

Códigos paralelos com python

Marco A. Zanata Alves – UFPR

Ambiente Google Colab

Google Colab

- O Colab permite que qualquer pessoa escreva e execute código Python arbitrário pelo navegador
- O ambiente é adequado para machine learning, análise de dados e educação.
 - Suporte para Python 2.7 e Python 3.6;
 - Aceleração de GPU grátis;
 - Bibliotecas pré-instaladas: Todas as principais bibliotecas Python, como o TensorFlow, o Scikit-learn, o Matplotlib;
 - Construído com base no Jupyter Notebook;



Limitações do colab

- Trata-se de um serviço de nuvem gratuito hospedado pelo Google
- Com um simples comando:

```
!cat /proc/cpuinfo
```

- Observamos que temos apenas 1 núcleo físico com 2 núcleos virtuais.
- Assim o paralelismo não vai nos dar grandes ganhos.

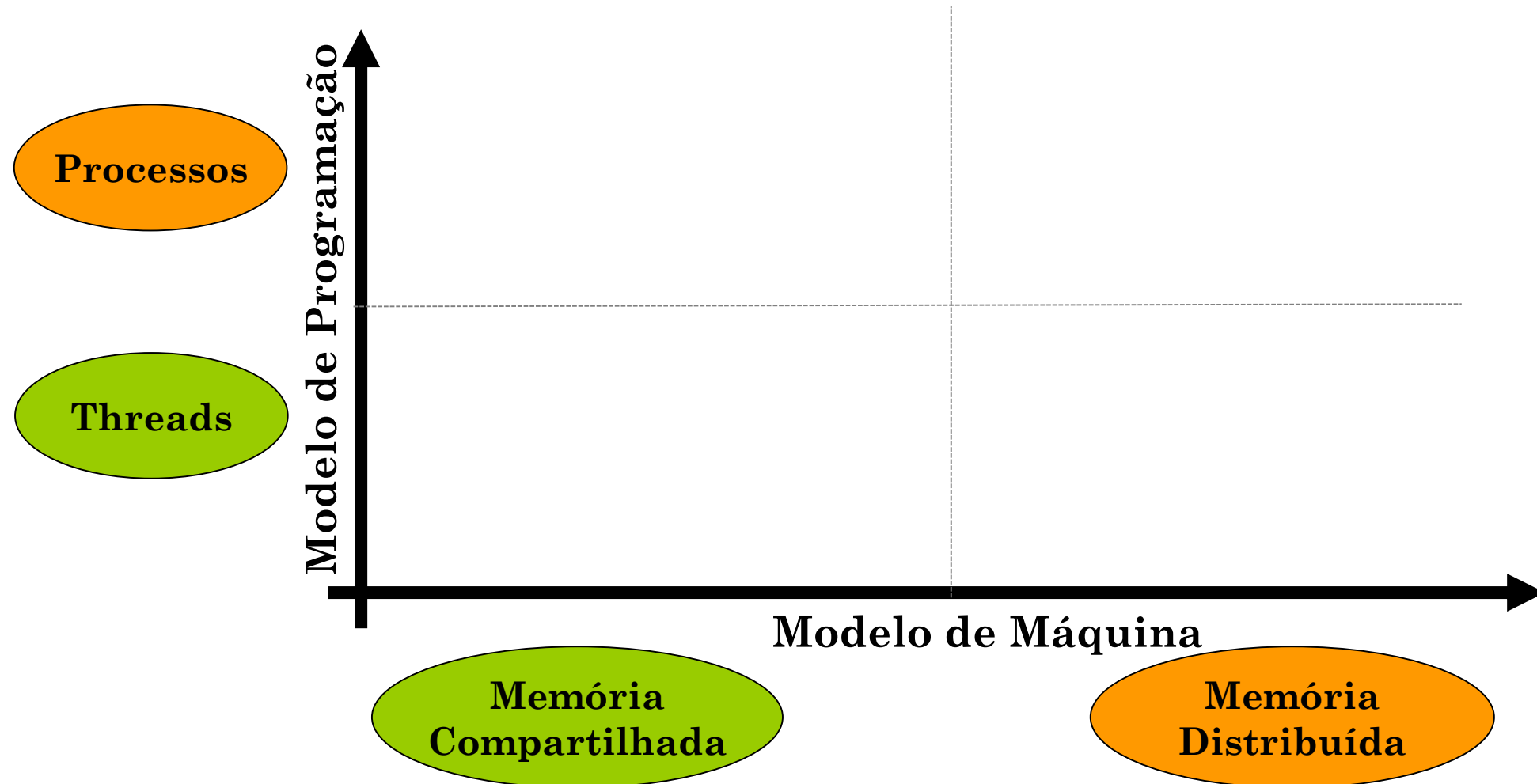
Obtendo informações sobre a máquina pelo python

```
import platform
import multiprocessing as mp

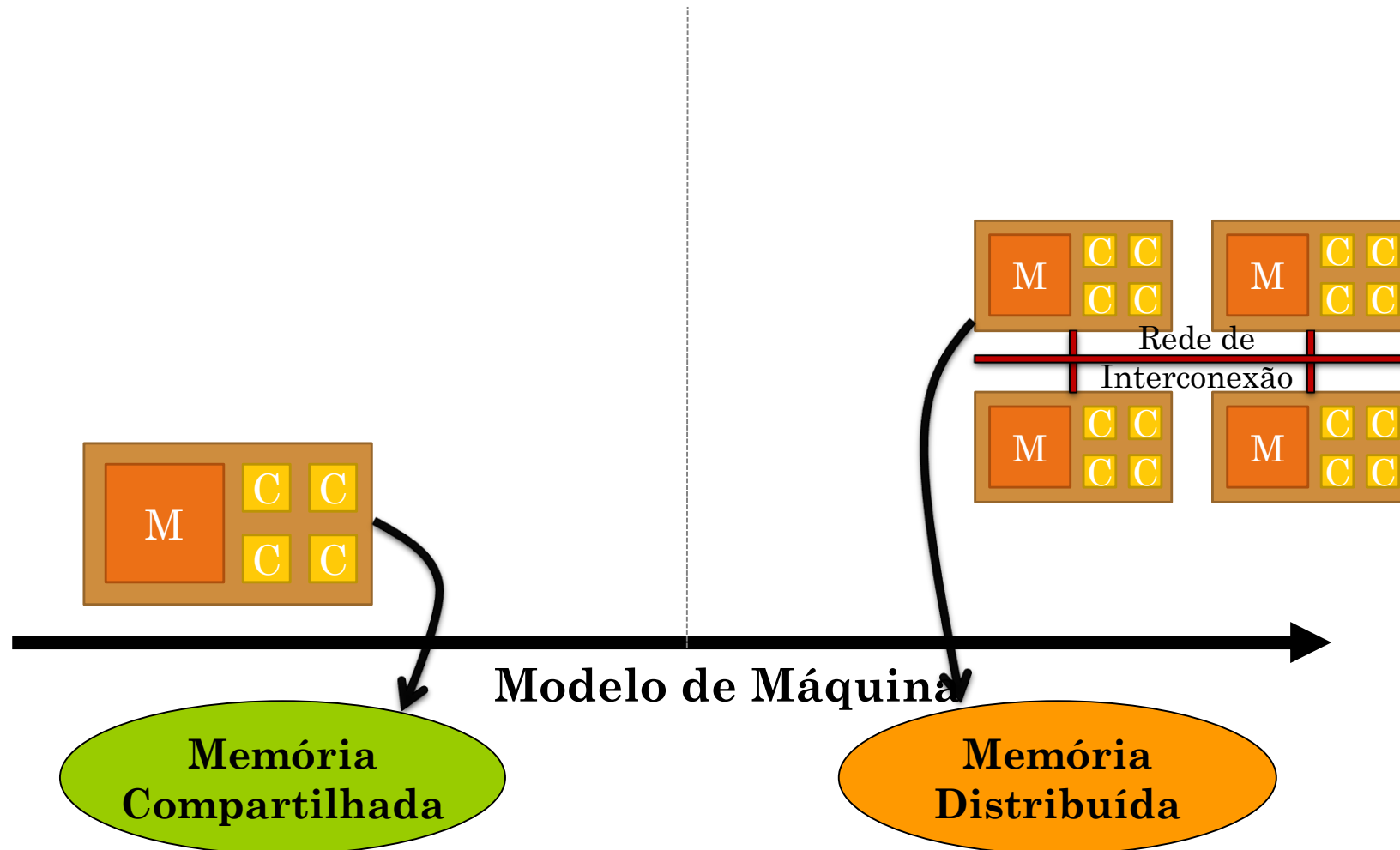
print('Python version  :', platform.python_version())
print('compiler         :', platform.python_compiler())
print('system           :', platform.system())
print('release          :', platform.release())
print('machine           :', platform.machine())
print('processor         :', platform.processor())
print('CPU count         :', mp.cpu_count())
print('interpreter:', platform.architecture()[0])
```

Modelo de programação vs. Modelo de máquina

Modelo de programação vs. Modelo de máquina



Modelo de programação vs. Modelo de máquina



Principais Modelos de Programação Paralela

- Programação em **Memória Compartilhada**

- Programação usando processos ou threads.
- Decomposição do domínio ou funcional com granularidade fina, média ou grossa.
- Comunicação através de **memória compartilhada**.
- Sincronização através de mecanismos de exclusão mútua.

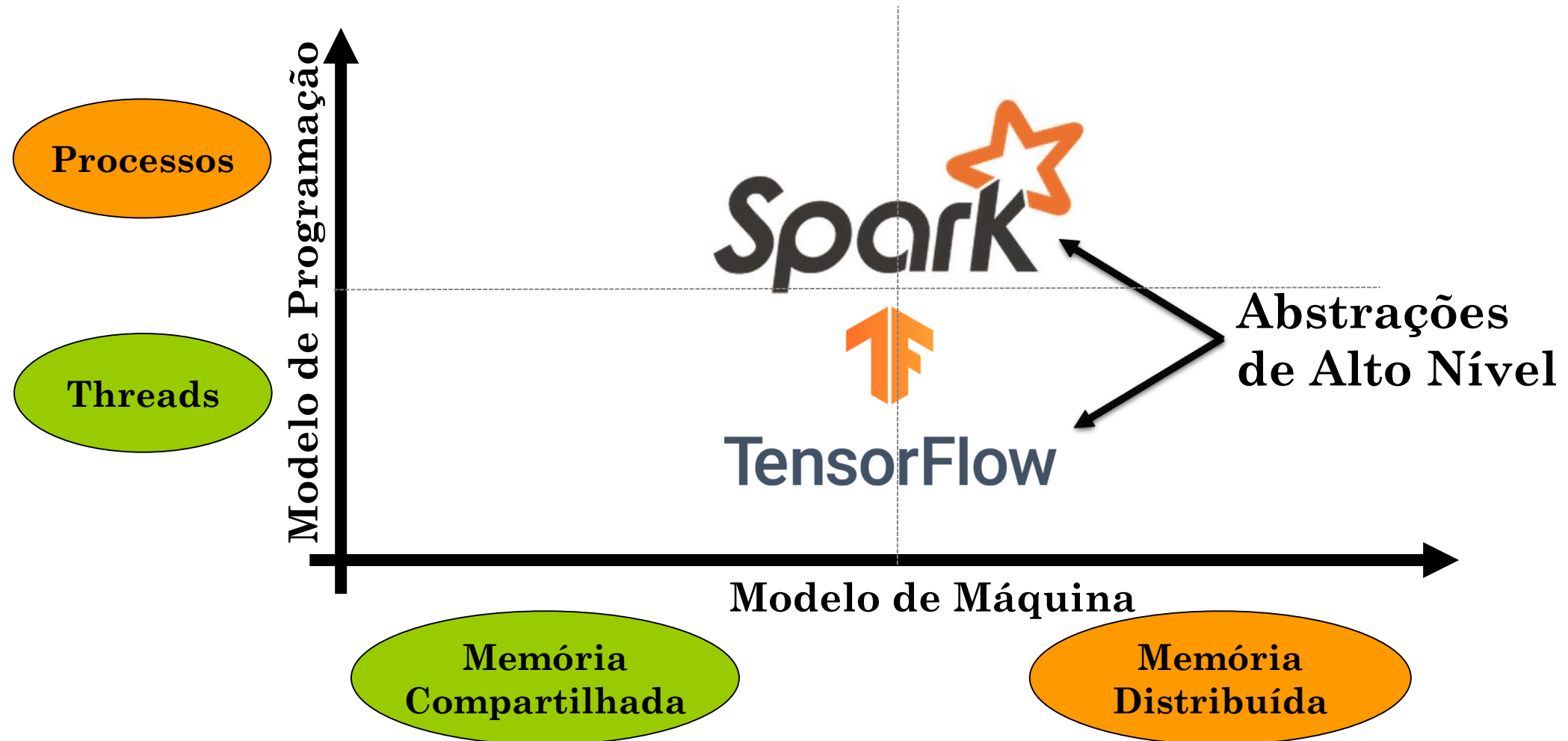


Nosso foco

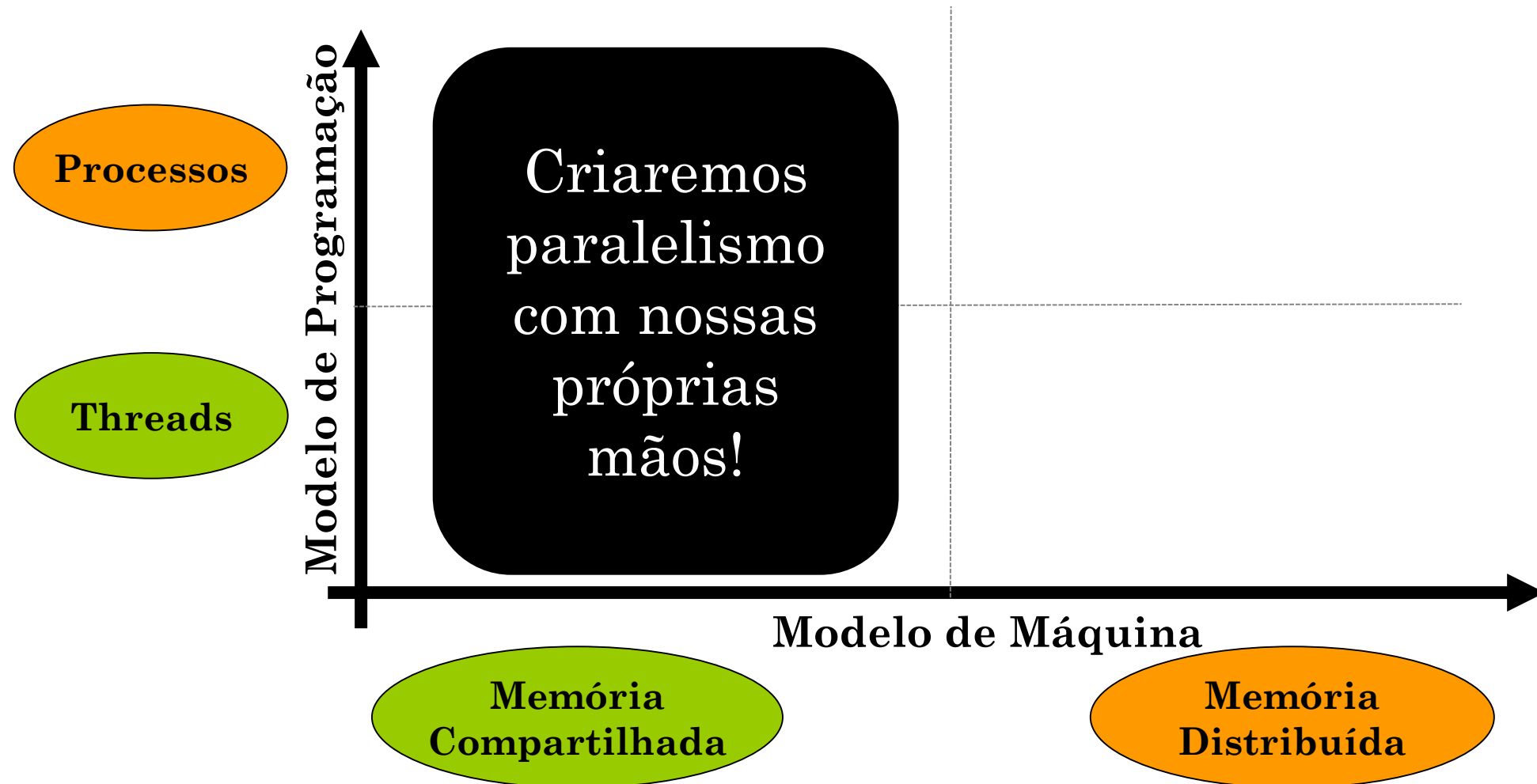
- Programação em Memória Distribuída

- Programação usando processos distribuídos
- Decomposição do domínio com granularidade grossa.
- Comunicação e sincronização por **troca de mensagens**.

Modelo de programação vs. Modelo de máquina

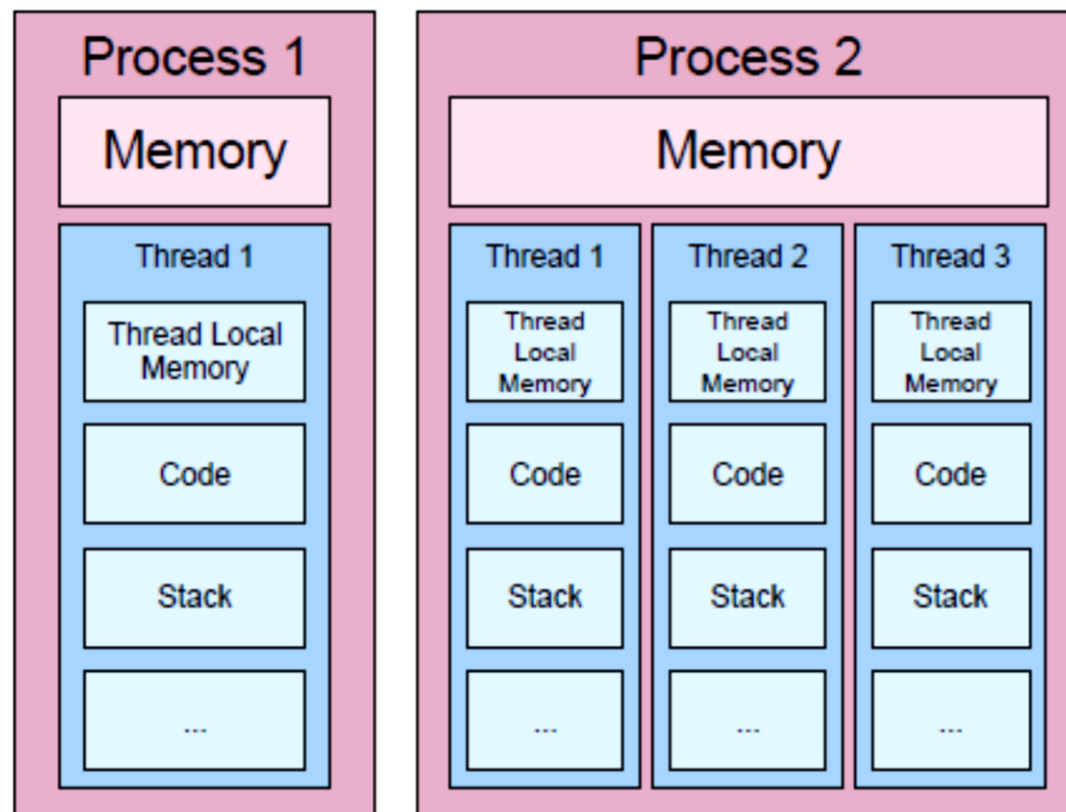


Modelo de programação vs. Modelo de máquina

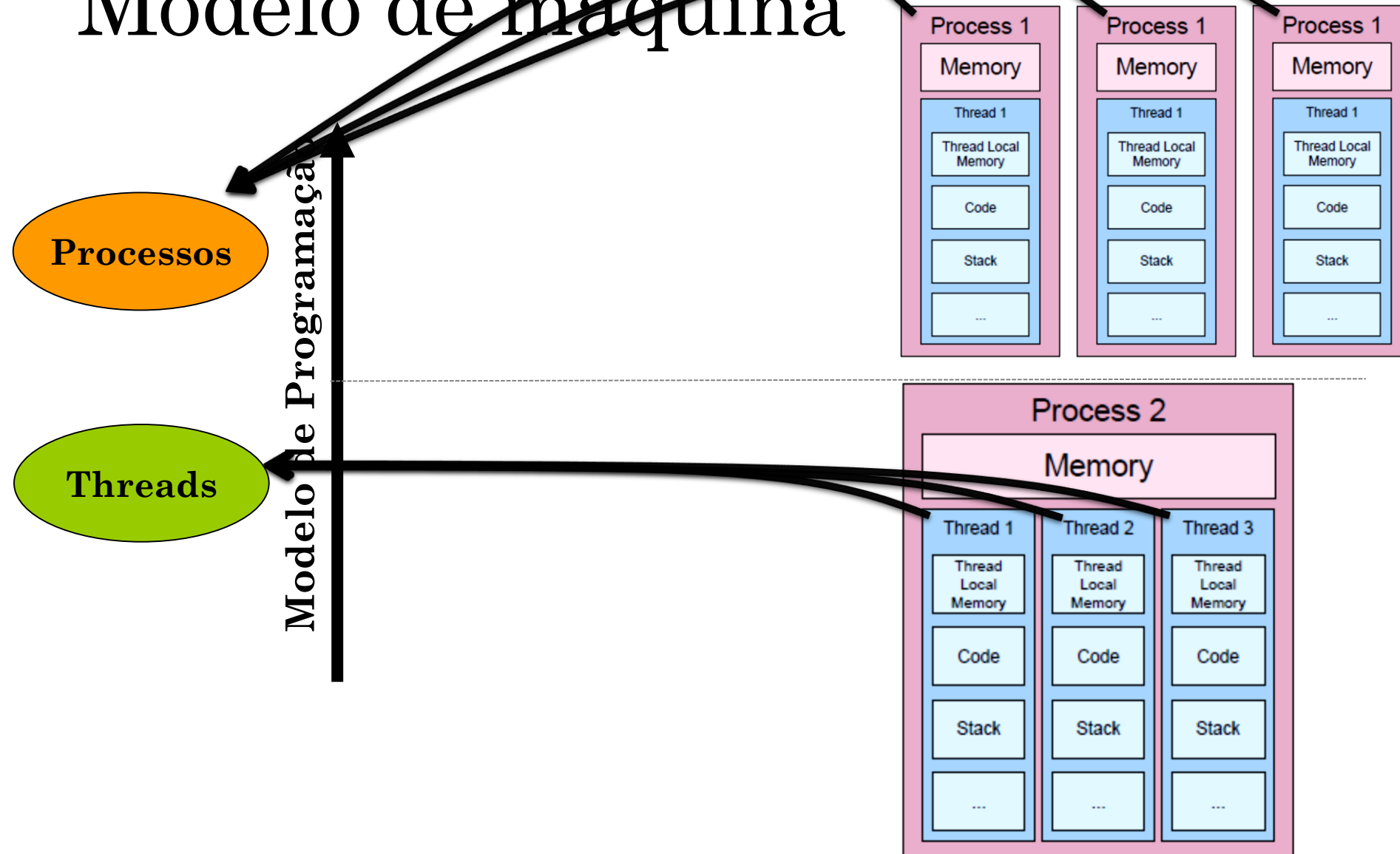


Dois tipos de tarefas: threads e processos

- Os processos possuem sua própria memória (variáveis, etc.)
- Um processo tem uma ou mais threads
- Threads compartilham a memória do processo ao qual pertencem
- Threads também são chamados de processos leves:
 - Elas são geradas mais rápido que processos
 - Trocas de contexto (se necessário) são mais rápidas



Modelo de programação vs. Modelo de máquina



Resumo:

- Basicamente você tem dois paradigmas:
- 1. Memória Compartilhada
 - Tarefas A e B compartilham alguma memória
 - Sempre que uma tarefa modifica uma variável na memória compartilhada, a(s) outra(s) tarefa(s) vê essa mudança imediatamente
- 2. Passagem de Mensagem
 - A tarefa A envia uma mensagem para a tarefa B
 - A tarefa B recebe a mensagem e faz algo com ela
- O primeiro paradigma é geralmente usado com threads e o último com processos.

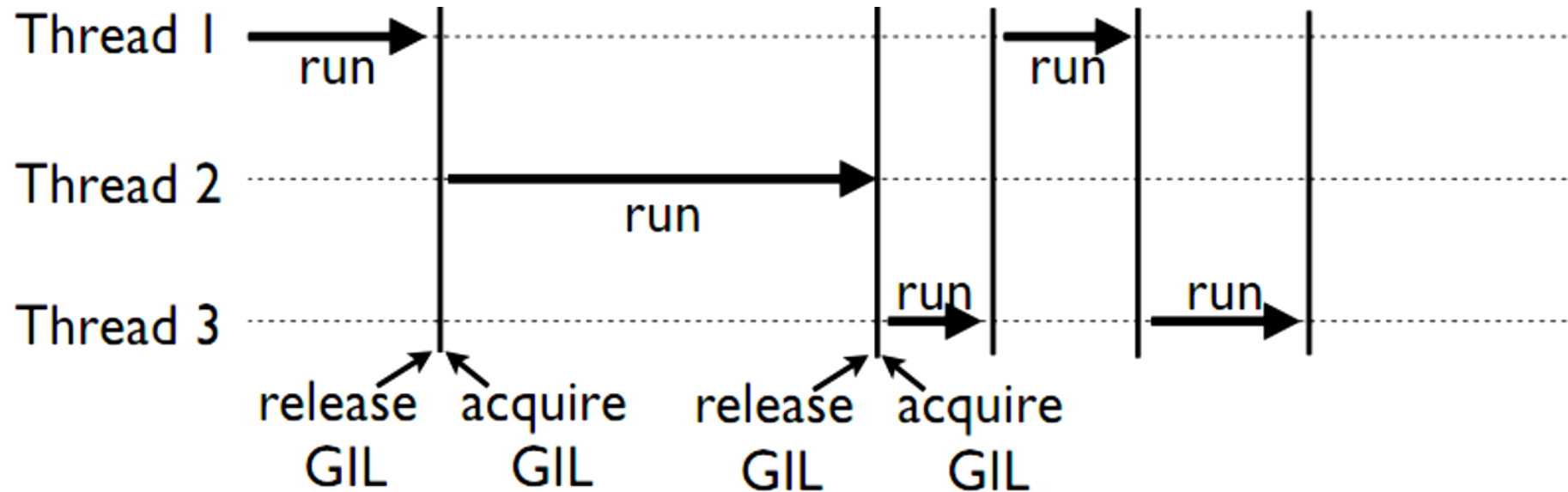
Global Interpreter Lock

GIL

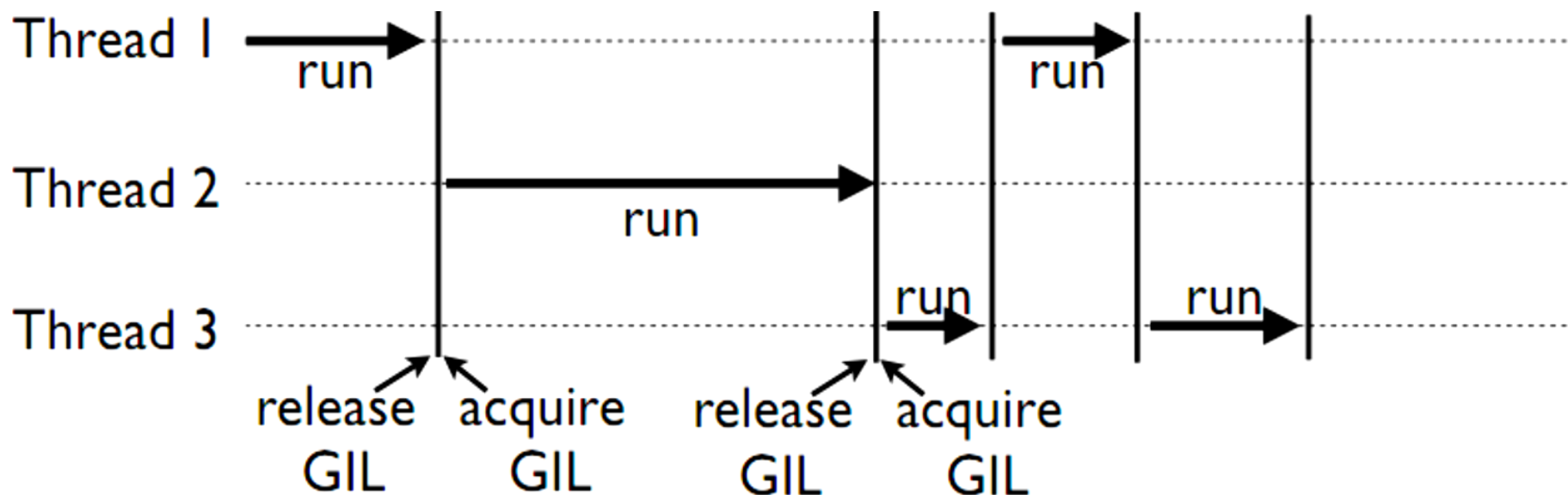
Segurança de Threads

- O interpretador padrão do Python foi projetado com a simplicidade em mente e possui um mecanismo seguro para thread, o chamado “GIL” (Global Interpreter Lock).
- Para evitar conflitos entre threads, ele executa apenas uma instrução por vez (o chamado processamento serial ou single-threading).

Como funciona a GIL



Como funciona a GIL



Na verdade, o GIL impede que os threads sejam executados em paralelo no Python!!!

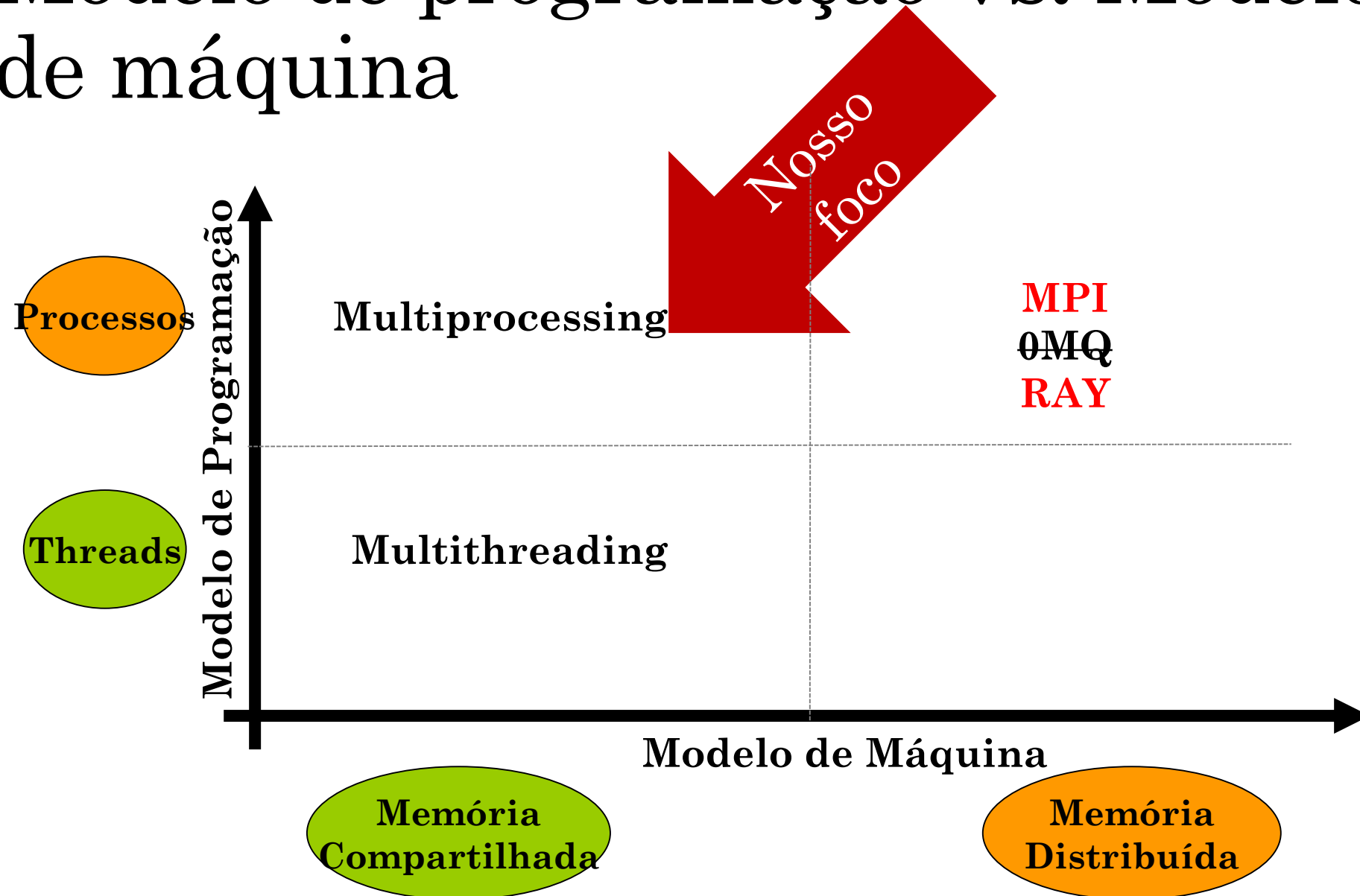
Como funciona a GIL

- Mas, não é tão ruim assim:
- Os desenvolvedores de módulos não precisam se preocupar com efeitos de maléficos de threads (ex. condições de corrida).
- Não é tão problemático assim ao executar vários threads de E/S (e.g. disco) simultaneamente.
- Geralmente, as extensões C (Cython) liberam o GIL durante a operação, permitindo usar várias threads simultaneamente.

Superando as limitações do GIL

- **Podemos usar vários processos completos** em vez de threads.
- Cada **processo tem seu próprio GIL, sem bloquear um ao outro.**
- A partir do python 2.6, a biblioteca padrão inclui um módulo de multiprocessing, com a mesma interface do módulo de threading.
- É possível compartilhar memória entre processos, incluindo matrizes numpy.
- Isso permite a maioria dos benefícios das threads sem os problemas do GIL.

Modelo de programação vs. Modelo de máquina



O módulo de multiprocessamento

O módulo de multiprocessamento

- O módulo de multiprocessamento tem muitos recursos poderosos.
- Use a documentação oficial como um ponto de partida.
- Vamos começar com uma breve visão geral de diferentes abordagens

Cuidados

- Você não tem **absolutamente nenhuma garantia** sobre o momento em que partes específicas de suas tarefas são executadas.
- Todas as variáveis de cada processo (escopo) **são visíveis apenas** dentro desse processo
- Com processos o **acesso a uma variável é explícito** (por exemplo, passando-a como um argumento para o processo ou via mensagem)

Exemplo ZERO

Hello world

```
from multiprocessing import Process

def f(name):
    print('hello, sou', name)

if __name__ == '__main__':
    p = Process(target=f, args=('bob filho', ))

    p.start()

    p.join()
```

Exemplo ZERO

Hello world

```
from multiprocessing import Process

def f(name):
    print('hello, sou', name)

if __name__ == '__main__':
    p = Process(target=f, args=('bob filho', ))

    p.start()

    p.join()
```



if `__name__ == '__main__':`

- Quando o interpretador Python lê um código fonte, ele primeiro define algumas variáveis especiais, como a **variável** `__name__`
- Se você o seu código for o **programa principal**, o interpretador fará `__name__ = "__main__"`
- Caso seu código esteja sendo **importado**, o interpretador fará `__name__ = "<nome do arquivo sem .py>"`

Exemplo

\$>

teste.py

```
print(__name__)
```

```
print("Programa A")
```

Exemplo

```
$> python teste.py  
__main__  
Programa A
```

```
teste.py  
  
print(__name__)  
print("Programa A")
```

Exemplo

\$>

teste2.py

```
import teste
```

```
print("Programa B")
```

teste.py

```
print(__name__)
```

```
print("Programa A")
```

Exemplo

```
$> python teste2.py
```

Programa A

`__teste__`

Programa B

teste2.py

```
import teste
```

```
print("Programa B")
```

teste.py

```
print(__name__)
```

```
print("Programa A")
```

Exemplo

```
$> python teste2.py  
Programa B
```

teste2.py

```
import teste
```

```
print("Programa B")
```

teste.py

```
if __name__ == '__main__':
```

```
    print(__name__)
```

```
    print("Programa A")
```



```
if __name__ == '__main__':
```

- Em um programa paralelo, apenas a thread/processo master executará o código dentro do IF.
- Se você não adicionar essa proteção, então você pode entrar um loop interminável de criação de novas threads/processos

if __name__ == '__main__':

```
# Script foo.py
```

```
if __name__ == '__main__':
```

```
    print('me executou pelo terminal')
```

```
else:
```

```
    print('me executou como um módulo')
```

```
$ python foo.py  
me executou pelo terminal
```

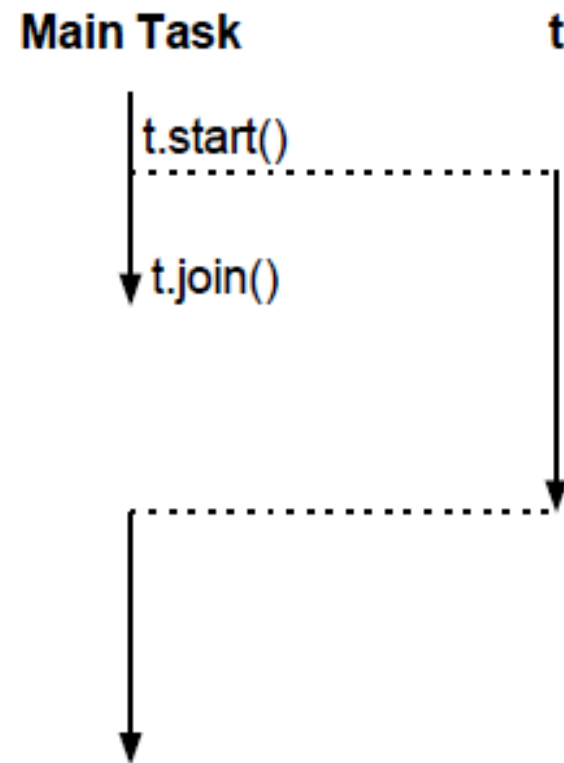
```
>>> import foo  
me executou como um módulo
```

Iniciando e terminando uma tarefa

- Uma tarefa é um programa ou método que é executado simultaneamente.

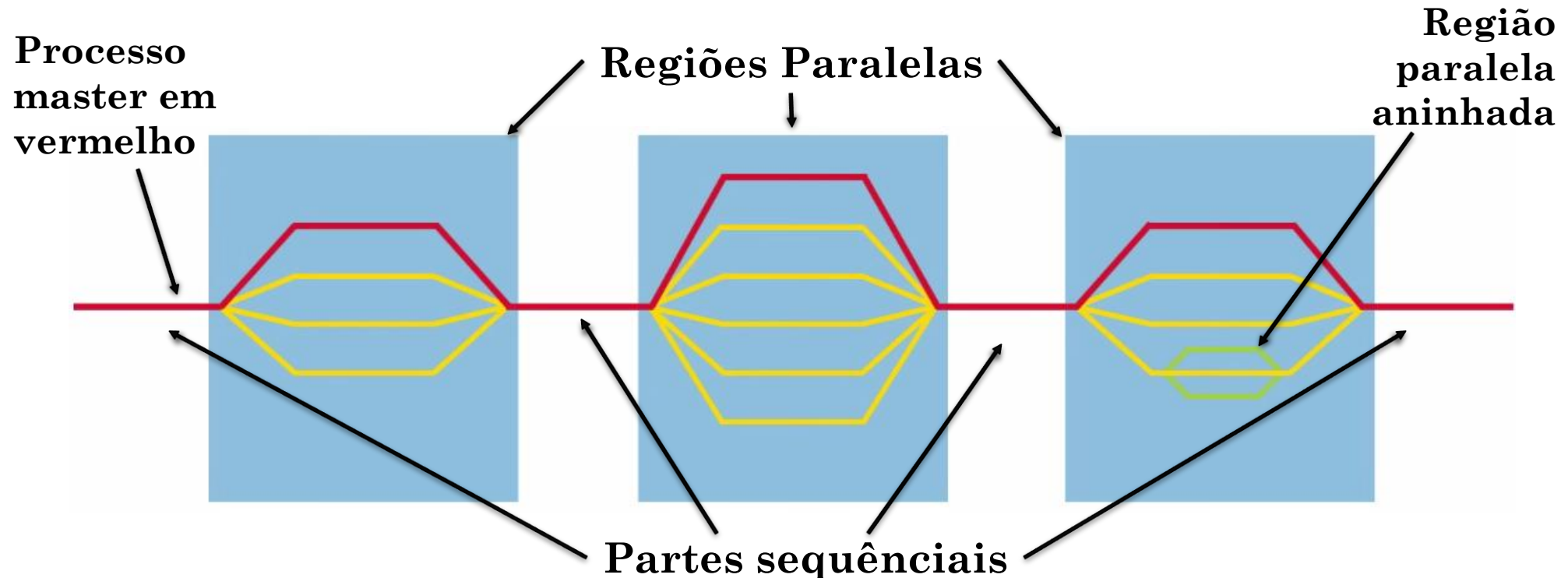
```
# Inicia a tarefa T
# T irá executar concorrentemente e
# (i.e. esse) programa irá continuar
T = Task()
T.start()
...
# Espera a tarefa T terminar
T.join()
```

Join sincroniza a tarefa pai com a tarefa filho, aguardando que a tarefa filho termine.



Modelo de Programação: Paralelismo Fork-Join

- A thread Master despeja um time de processos-filho como necessário
- Paralelismo é adicionado aos poucos até que o objetivo de desempenho é alcançado. Ou seja o programa sequencial evolui para um programa paralelo



Exemplo ZERO

Hello world

```
from multiprocessing import Process

def f(name):
    print('hello, sou', name)

if __name__ == '__main__':
    p = Process(target=f, args=('bob filho', ))

    p.start()

    print('hello, sou', 'bob pai') ## Temos 2 processos!

    p.join()
```

Exemplo ZERO

4x Hello world

```
from multiprocessing import Process

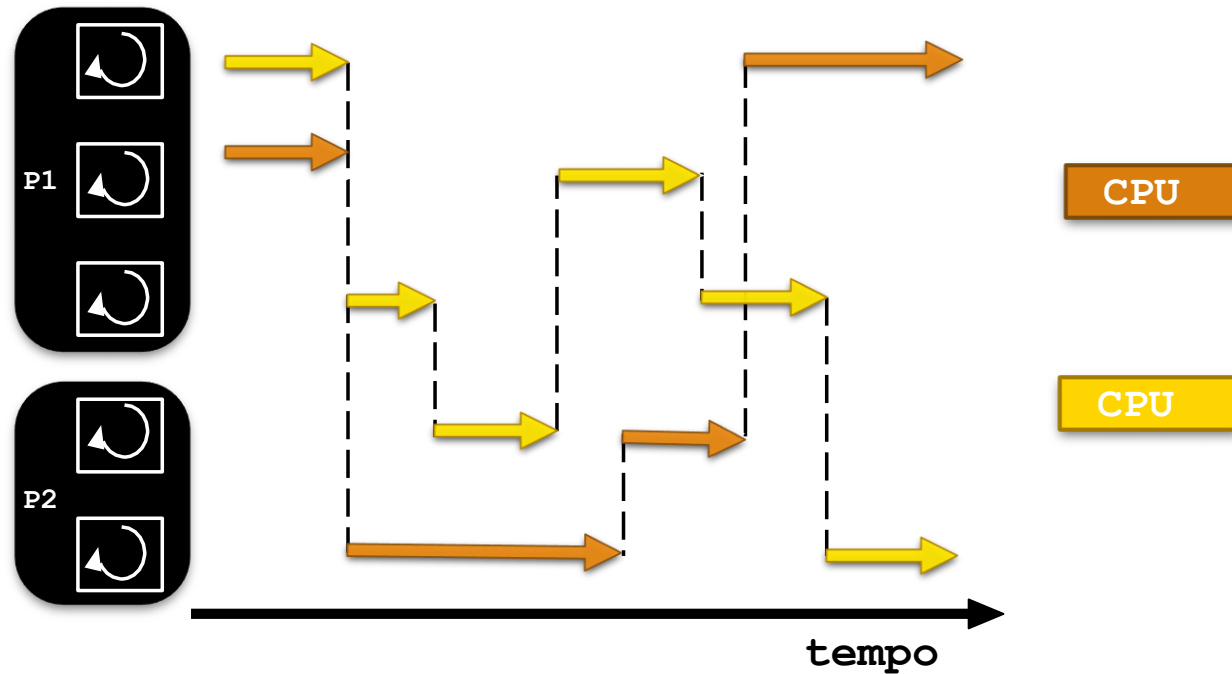
def f(name, id):
    print('hello, sou', name, id)

if __name__ == '__main__':
    procs = []
    for i in range(4):
        p = Process(target=f, args=('bob filho', i, ))
        procs.append(p)

    print('hello, sou bob pai')
    for i in range(4):
        procs[i].start()
    for i in range(4):
        procs[i].join()
```

Execução de múltiplos Processos

- Todos os threads de um processo podem ser executados concorrentemente e em diferentes processadores, caso existam.



Exemplo ZERO

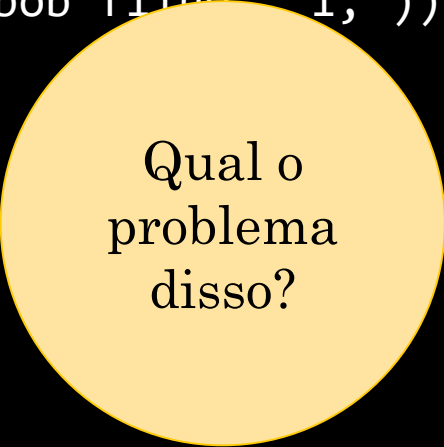
4x Hello world

```
from multiprocessing import Process

def f(name, id):
    print('hello, sou', name, id)

if __name__ == '__main__':
    procs = []
    for i in range(4):
        p = Process(target=f, args=('bob filho', i, ))
        procs.append(p)

    print('hello, sou bob pai')
    for i in range(4):
        procs[i].start()
        procs[i].join()
```



Qual o problema disso?

Exemplo ZERO

4x Hello world

```
from multiprocessing import Process
```

```
def f(name, id):  
    print('hello, sou', name, id)
```

```
if __name__ == '__main__':  
    procs = []  
    for i in range(4):  
        p = Process(target=f, args=('bob f', i))  
        procs.append(p)
```

```
print('hello, sou bob pai')  
for i in range(4):  
    procs[i].start()  
    procs[i].join()
```

A execução
virou
sequencial!

Qual o
problema
disso?

Processo
Master

start

join

start

join

start

join

⋮

Processo
Filho 0

Processo
Filho 1

Processo
Filho 2

Exemplo ZERO

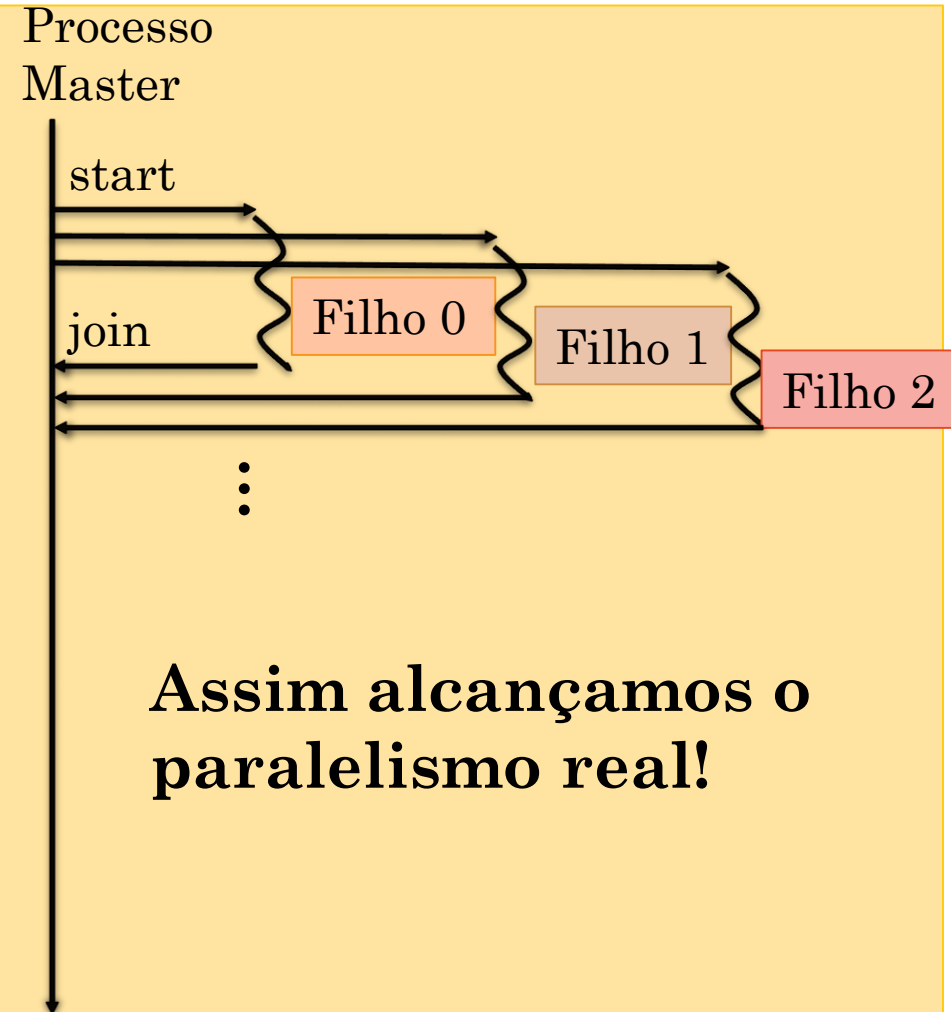
4x Hello world

```
from multiprocessing import Process

def f(name, id):
    print('hello, sou', name, id)

if __name__ == '__main__':
    procs = []
    for i in range(4):
        p = Process(target=f, args=('bob filho',
                                   procs.append(p)

    print('hello, sou bob pai')
    for i in range(4):
        procs[i].start()
    for i in range(4):
        procs[i].join()
```



Exemplo ZERO

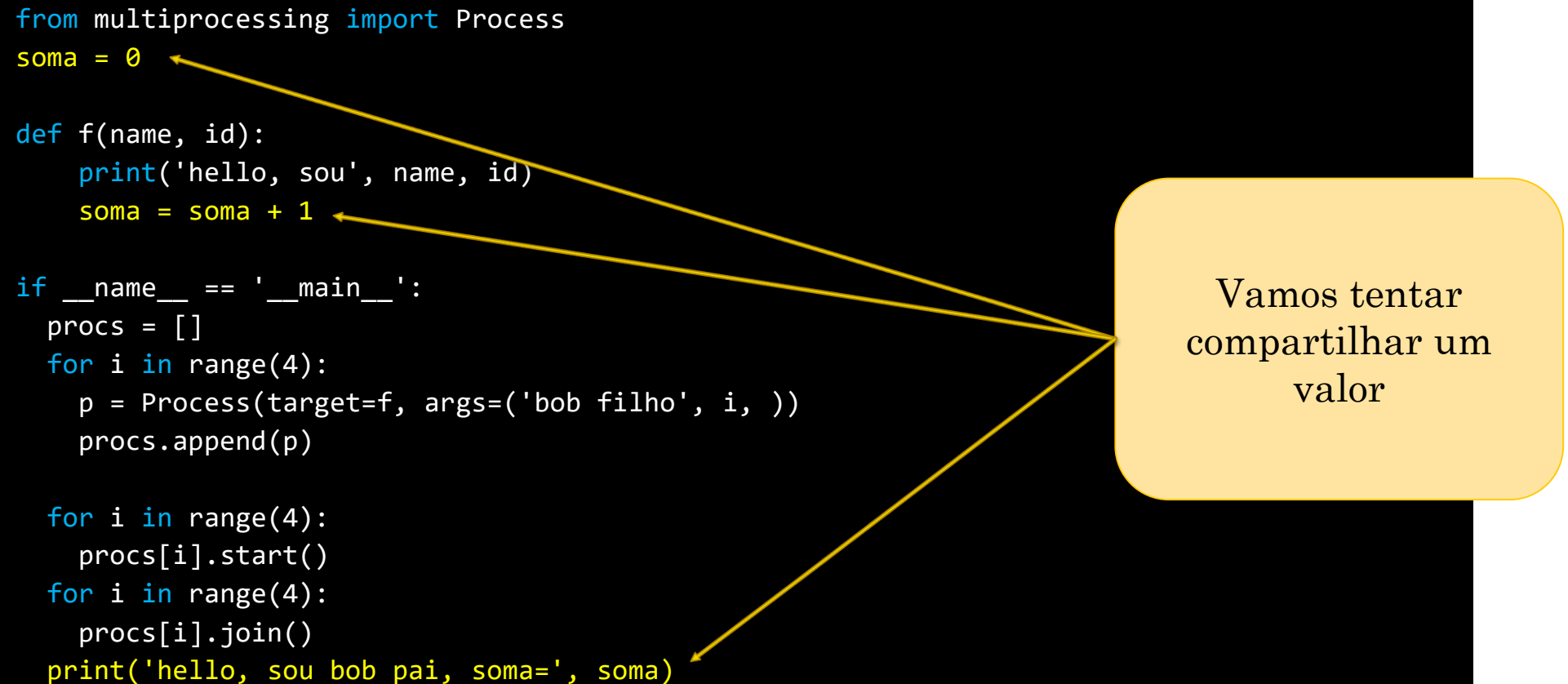
4x Hello world

```
from multiprocessing import Process
soma = 0

def f(name, id):
    print('hello, sou', name, id)
    soma = soma + 1

if __name__ == '__main__':
    procs = []
    for i in range(4):
        p = Process(target=f, args=('bob filho', i, ))
        procs.append(p)

    for i in range(4):
        procs[i].start()
    for i in range(4):
        procs[i].join()
    print('hello, sou bob pai, soma=', soma)
```



Vamos tentar
compartilhar um
valor

Exemplo ZERO

4x Hello world

```
from multiprocessing import Process
soma = 0

def f(name, id, soma):
    print('hello, sou', name, id)
    soma = soma + 1

if __name__ == '__main__':
    procs = []
    for i in range(4):
        p = Process(target=f, args=('bob filho', i, soma, ))
        procs.append(p)

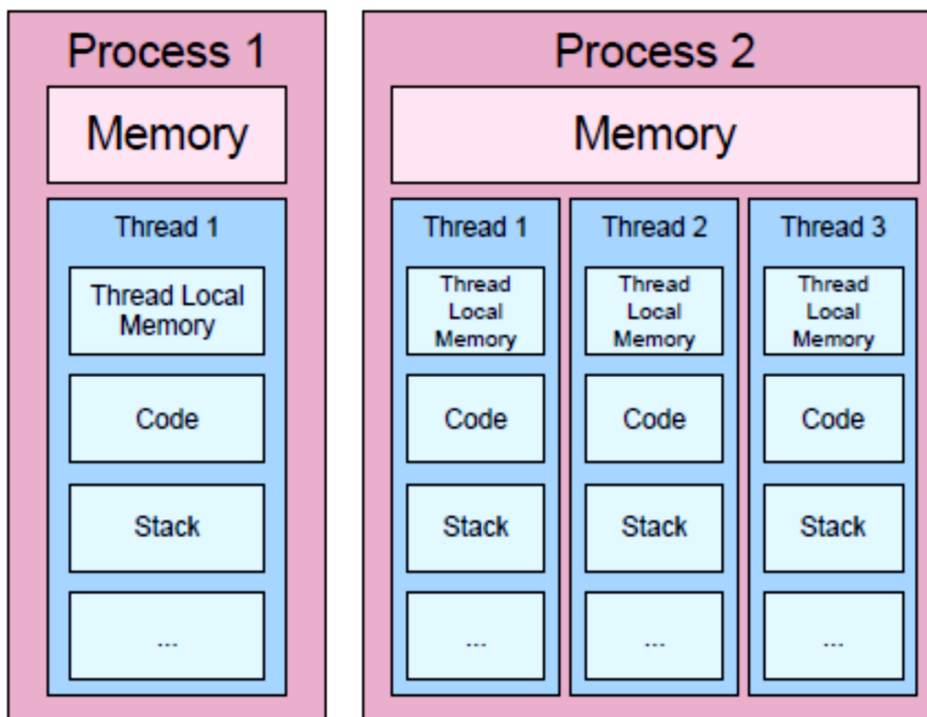
    for i in range(4):
        procs[i].start()
    for i in range(4):
        procs[i].join()
    print('hello, sou bob pai, soma =', soma)
```

Vamos tentar novamente



Cópia do ambiente/variáveis

- Durante a criação de um processo:
- 1) **Todo o ambiente será copiado para o novo processo** (programa no estado atual)
- 2) Iniciará a execução dentro da função escolhida.
- Dessa maneira, as variáveis pré-existentes **são copiadas**.
- **Quando um processo alterar alguma de suas variáveis, ele estará alterando sua cópia local.**



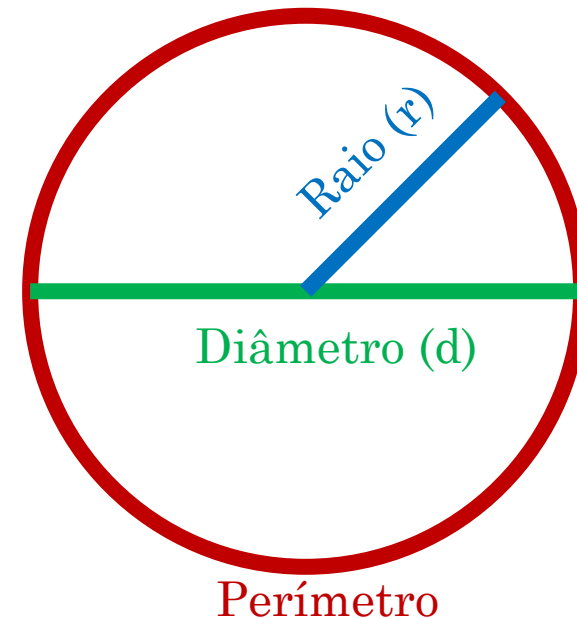
Exemplo: cálculo de Pi

O número π

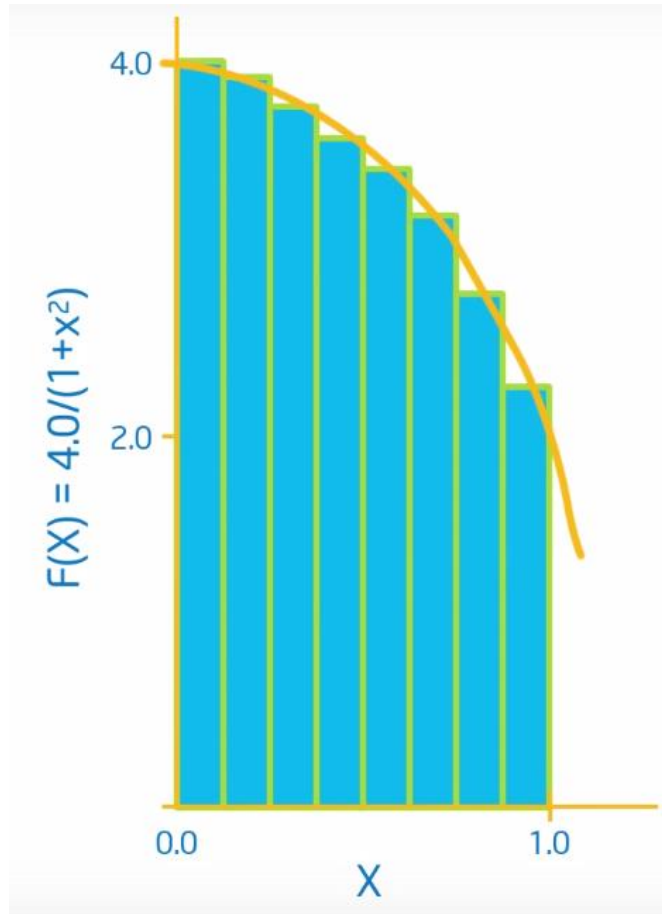
- O número π (pi) é uma proporção numérica
- Definido pela relação entre o perímetro e diâmetro de uma circunferência

$$\pi = \frac{p}{d}$$

$$\begin{aligned} \text{Diâmetro} &= 2 \cdot r \\ \text{Perímetro} &= 2\pi \cdot r \\ \text{Área} &= \pi \cdot r^2 \end{aligned}$$



Exercício Integração numérica



- Matematicamente, sabemos que:

$$\int_0^1 \frac{4.0}{1+x^2} dx = \pi$$

- Podemos aproximar essa integral como a soma de retângulos:

$$\sum_{i=0}^n F(x_i) \Delta x \cong \pi$$

- Onde cada retângulo tem largura Δx e altura $F(x_i)$ no meio do intervalo i .

Melhorias possíveis

- Diversas melhorias são possíveis nesse código.
- Podemos tentar utilizar espaço linear, outras formas de laço, etc.
- **Mas esse exemplo é uma carga de trabalho ótima para testarmos a programação paralela.**

Pi Sequential

```
import time
def pi_naive(start, end, step):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    return sum

if __name__ == "__main__":
    num_steps = 100_000_000 #100.000.000 (10+e8) = 8 seg. (2024)
    sums = 0.0
    step = 1.0/num_steps
    tic = time.time() # Tempo Inicial
    sums = pi_naive(0, num_steps, step)
    toc = time.time() # Tempo Final
    pi = step * sums
    print ("Valor Pi: %.10f" %pi)
    print ("Tempo Pi: %.8f s" %(toc-tic))
```

Paralelizando o pi

Passos para paralelizar

- 1. Criar P processos paralelos
- 2. Cada processo deve processar uma parcela do trabalho $\left(\frac{N}{P}\right)$
- 3. Devemos unir os resultados parciais
- 4. Apresentamos o resultado final

Dúvidas sobre como dividir o trabalho entre os processos

1. Quantas threads irei criar?
 - Uma boa ideia é utilizar o mesmo número de núcleos (ou menos).
2. Como irei dividir os laços de repetições entre os processos?
 - Precisamos calcular quantas e quais iterações cada processo irá executar
3. Quais parâmetros irei passar aos processos?
 - Podemos quebrar o linear space e enviar aos processos
(Péssimo para o desempenho, pense nos dados enviados).
 - Podemos enviar para cada processo, onde iniciar e onde terminar, assim dentro do processo, cada um cria seu linearspace ou seu laço de repetição.
(Ótimo para o desempenho, quase zero dados enviados)

Dividindo o domínio

Execução
Sequencial

Processo 0



Dividindo o domínio

**Execução
Sequencial**

Processo 0



Divisão Linear

Processo 0

Processo 1

Processo 2

Processo 3



Dividindo o domínio

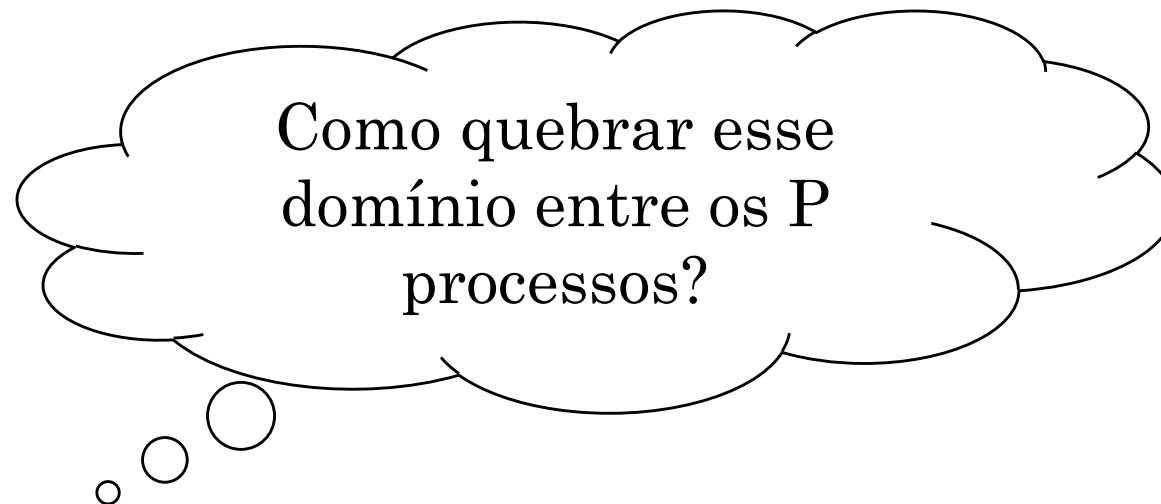


Divisão Linear

Código Sequencial

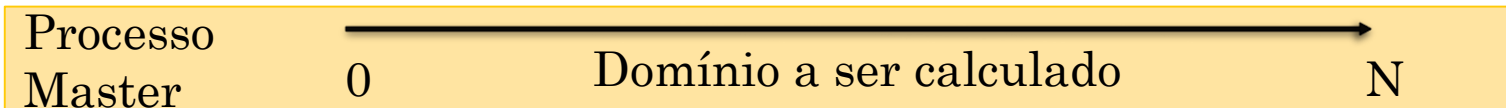


Código Paralelo

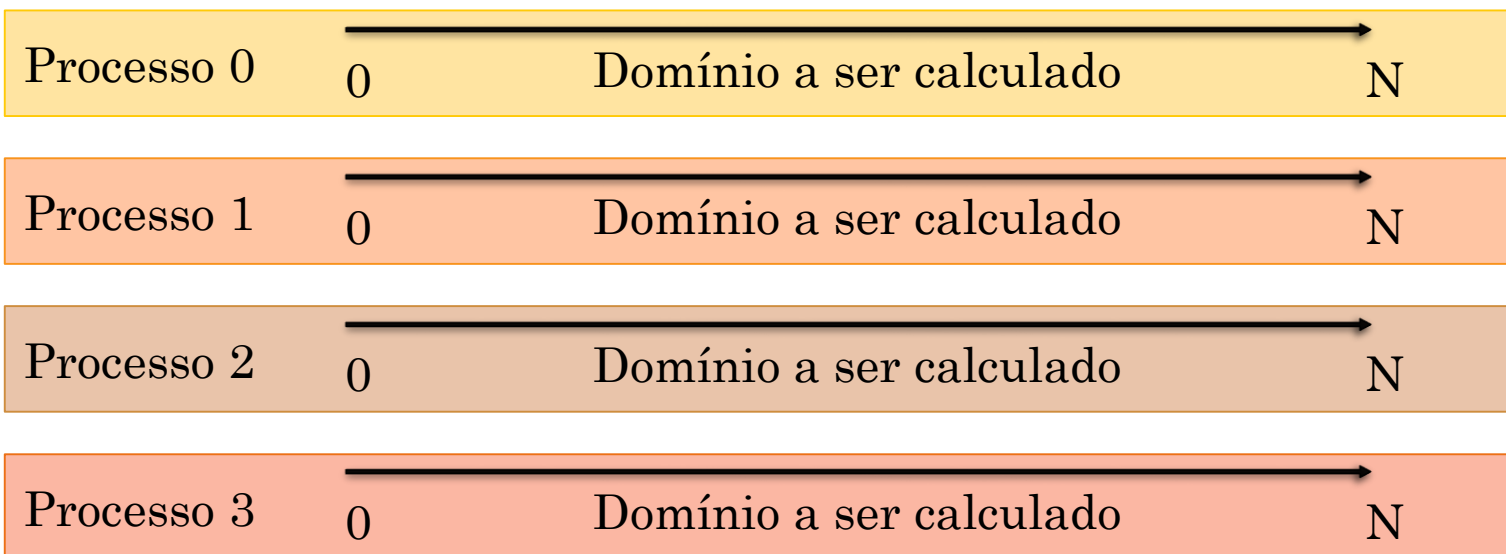


Divisão Linear

Código Sequencial



Código Paralelo



Divisão Linear

Código Sequencial

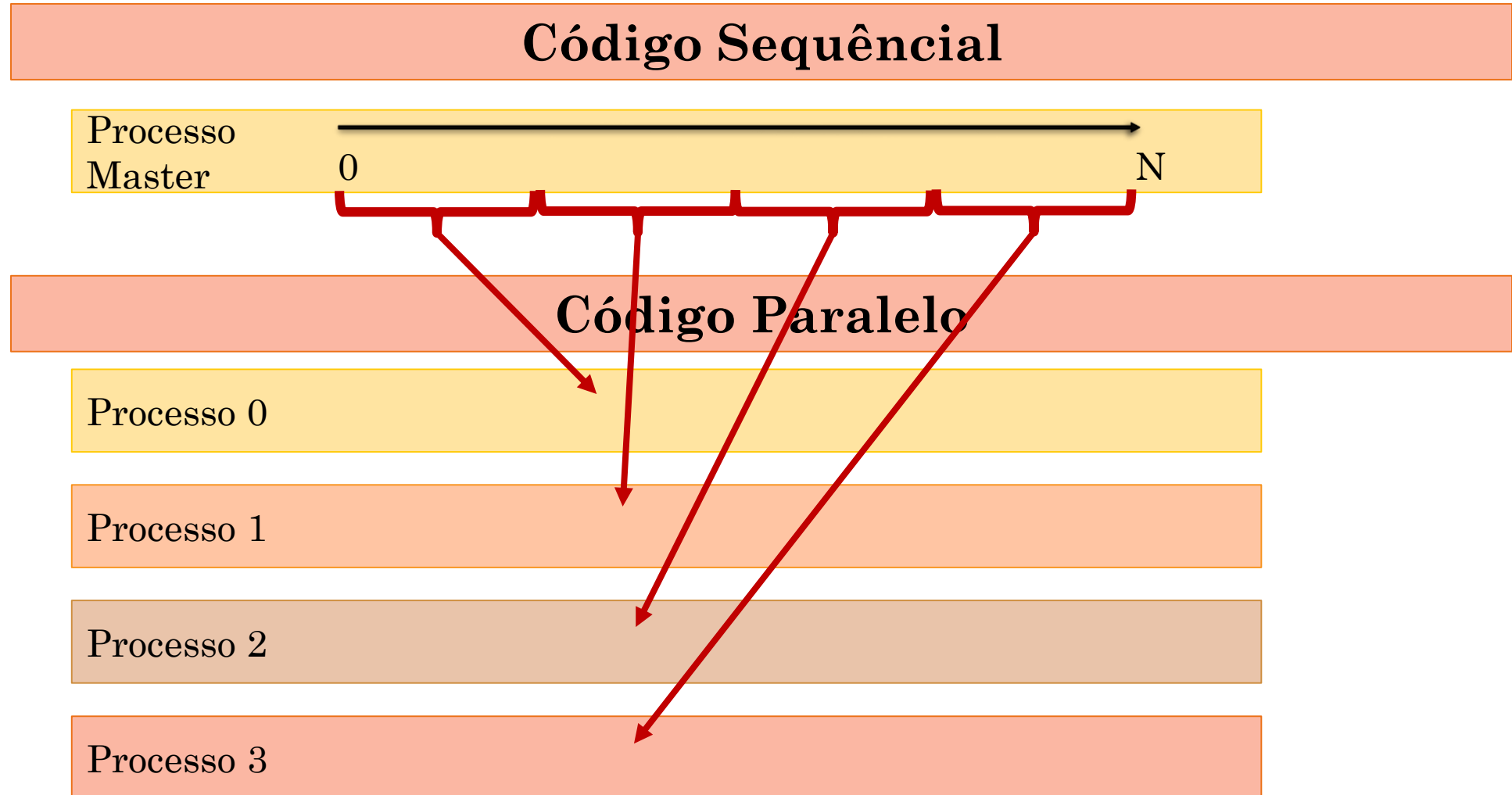


Código Paralelo

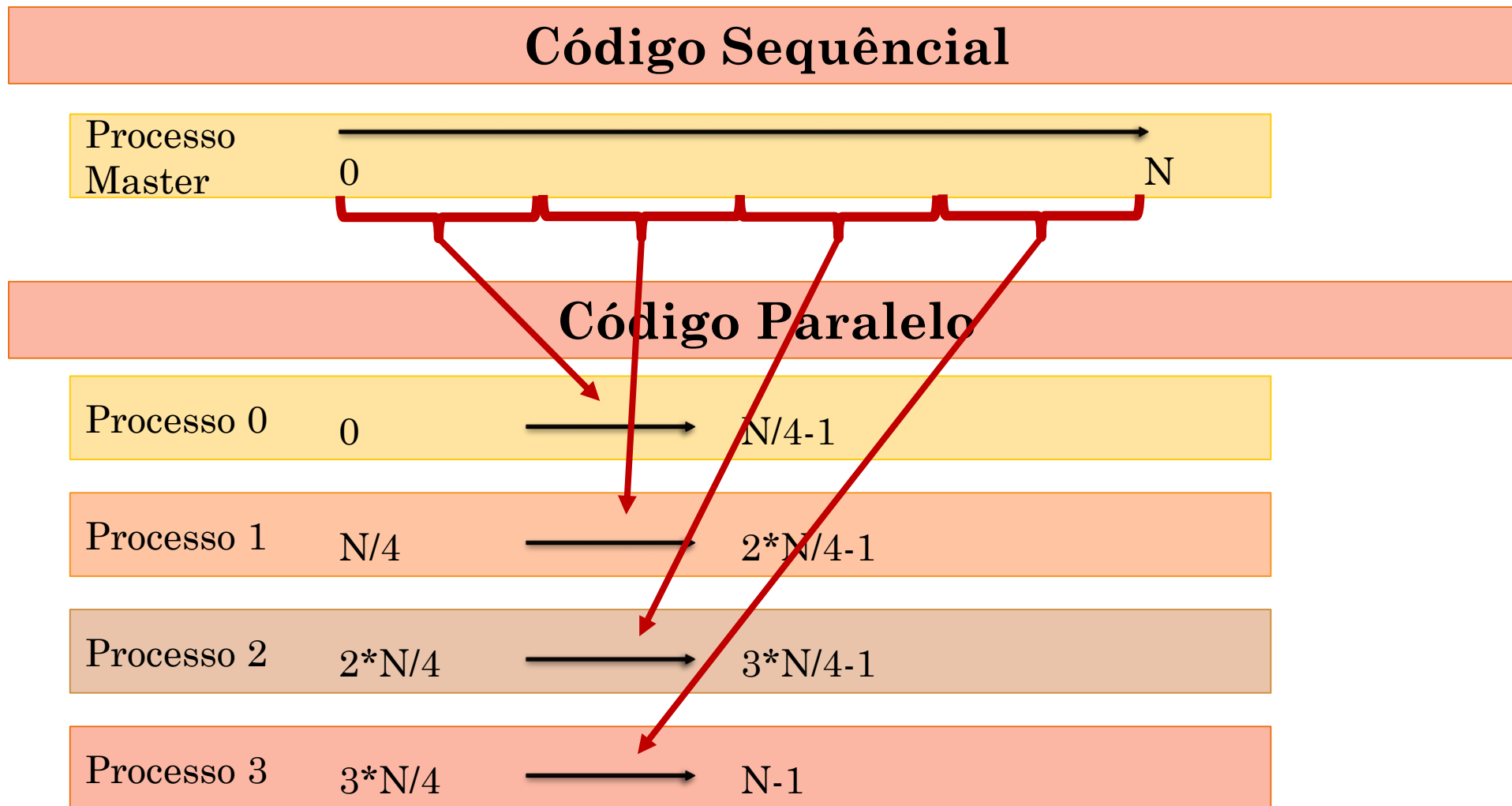


Péssimo! Todos os processos estão repetindo o mesmo cálculo!!!

Divisão Linear



Divisão Linear



Divisão Linear

Código Paralelo

Processo 0 0 $\longrightarrow$ $N/4-1$

Processo 1 $N/4$ $\longrightarrow$ $2*N/4-1$

Processo 2 $2*N/4$ $\longrightarrow$ $3*N/4-1$

Processo 3 $3*N/4$ $\longrightarrow$ $N-1$

Legal, agora
precisamos
sistematizar
essa divisão!

Divisão Linear



Código Paralelo



Divisão Linear



Código Paralelo

!

Processo 0 $0 * \frac{N}{4}$ $\longrightarrow$ $(0 + 1) * \frac{N}{4}$

Processo 1 $1 * \frac{N}{4}$ $\longrightarrow$ $(1 + 1) * \frac{N}{4}$

Divisão Linear

Código Paralelo

Processo 0 $0 * \frac{N}{4}$ $\longrightarrow$ $(0 + 1) * \frac{N}{4} - 1$

Processo 1 $1 * \frac{N}{4}$ $\longrightarrow$ $(1 + 1) * \frac{N}{4} - 1$

Divisão Linear

Código Paralelo

Processo 0 $0 * \frac{N}{4}$ $\longrightarrow$ $(0 + 1) * \frac{N}{4} - 1$

Processo 1 $1 * \frac{N}{4}$ $\longrightarrow$ $(1 + 1) * \frac{N}{4} - 1$

Processo 2 $2 * \frac{N}{4}$ $\longrightarrow$ $(2 + 1) * \frac{N}{4} - 1$

Processo 3 $3 * \frac{N}{4}$ $\longrightarrow$ $(3 + 1) * \frac{N}{4} - 1$

Divisão Linear

Para i-ésimo
processo

$$Id * \frac{N}{Procs} \longrightarrow (Id + 1) * \frac{N}{Procs} - 1$$

Limite do domínio

número do processo

número total de processo

Código Paralelo

Processo 0

$$0 * \frac{N}{4} \longrightarrow (0 + 1) * \frac{N}{4} - 1$$

Processo 1

$$1 * \frac{N}{4} \longrightarrow (1 + 1) * \frac{N}{4} - 1$$

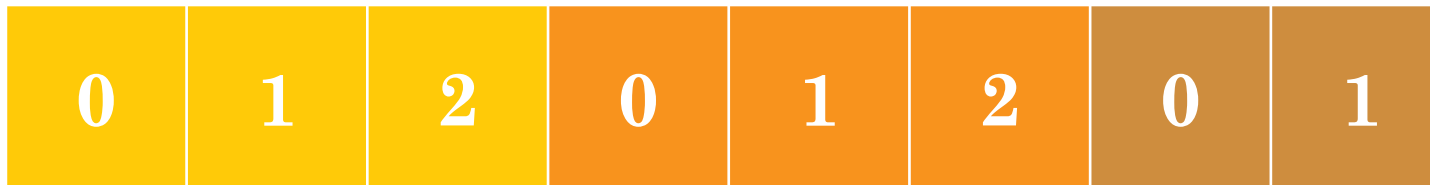
Processo 2

$$2 * \frac{N}{4} \longrightarrow (2 + 1) * \frac{N}{4} - 1$$

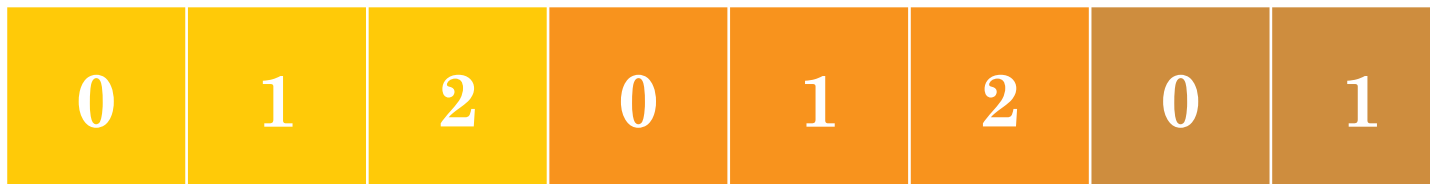
Processo 3

$$3 * \frac{N}{4} \longrightarrow (3 + 1) * \frac{N}{4} - 1$$

Divisão linear para **divisões inexatas**

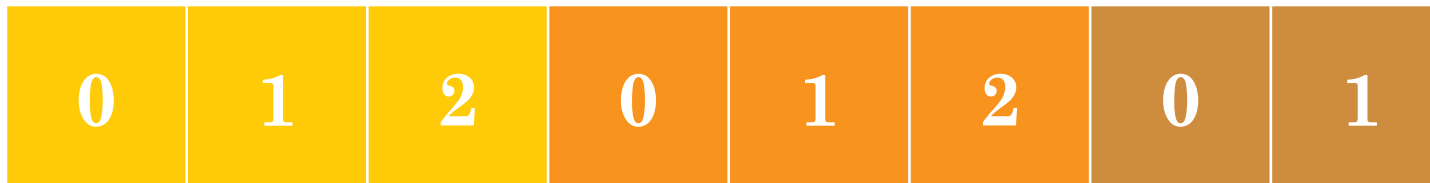


Divisão linear para divisões inexatas



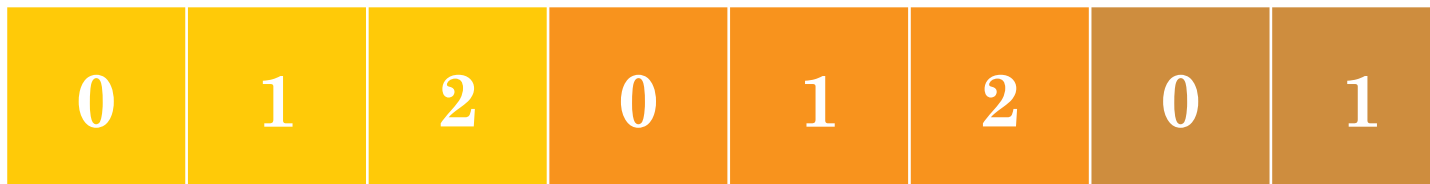
```
// divide por threads (arredondando para cima)  
fatia = (num_steps + nprocess - 1) // nprocess
```

Divisão linear para divisões inexatas



```
// divide por threads (arredondando para cima)  
fatia = (num_steps + nprocess - 1) // nprocess  
elem_inicial = th_id * fatia
```

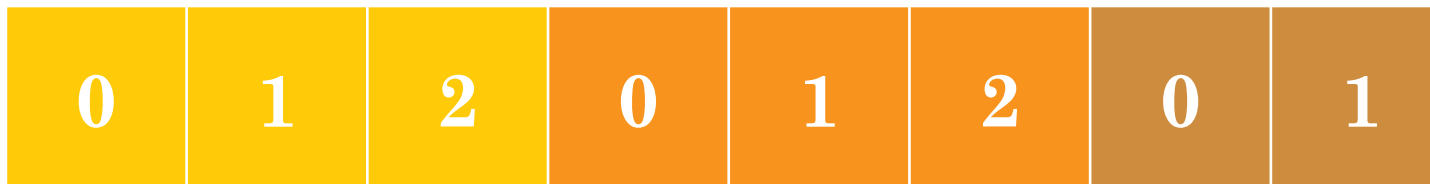
Divisão linear para divisões inexatas



```
// divide por threads (arredondando para cima)
// suporta divisões inexatas entre tasks e threads
fatia = (num_steps + nprocess - 1) // nprocess
elem_inicial = th_id * fatia
elem_final = min(elem_inicial + fatia, num_steps)
```



Divisão linear para divisões inexatas



```
// divide por threads (arredondando para cima)
// suporta divisões inexatas entre tasks e threads
fatia = (num_steps + nprocess - 1) // nprocess
elem_inicial = th_id * fatia
elem_final = min(elem_inicial + fatia, num_steps)

for i in range(elem_inicial, elem_final):
```


Exercício 1

- Transforme o programa Pi em paralelo!
 - Tudo bem que **não sabemos ainda como juntar os resultados**
 - Cada processo imprime na tela sua parcial (**a soma será manual**).
-
- **Vamos precisar:**
 1. Criar processos filhos que chamem a função de cálculo de pi.
 2. Para cada filho, teremos que enviar parâmetros
 3. Os parâmetros vão indicar onde cada filho começa e termina seu trabalho.

Exemplo para auxiliar...

```
from multiprocessing import Process

def f(name, id):
    print('hello, sou', name, id)

if __name__ == '__main__':
    procs = []
    for i in range(4):
        p = Process(target=f, args=('bob filho', i, ))
        procs.append(p)

    print('hello, sou', 'bob pai')
    for i in range(4):
        procs[i].start()
    for i in range(4):
        procs[i].join()
```

Pi paralelo

```
from multiprocessing import Process
import time
PROCS = 2
```

```
def pi(start, end, step):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    print(sum)
```

```
if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
    proc_size = num_steps / PROCS
```

```
    tic = time.time()
    workers = []
```

```
    for i in range(PROCS):
        worker = Process(target=pi, args=(i*proc_size, (i+1)*proc_size - 1, step, ))
        workers.append(worker)
```

```
    for worker in workers :
        worker.start()
    for worker in workers :
        worker.join()
    toc = time.time()

    print ("Tempo Pi: %.8f s" %(toc-tic))
```

Parâmetros
para a
função

Pi paralelo

```
from multiprocessing import Process
import time
PROCS = 2
def pi(start, end, step):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    print(sum)
if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
    proc_size = num_steps / PROCS

    tic = time.time()
    workers = []
    for i in range(PROCS):
        worker = Process(target=pi, args=(i*proc_size, (i+1)*proc_size - 1, step, ))
        workers.append(worker)
```

```
for worker in workers :
    worker.start()
for worker in workers :
    worker.join()
toc = time.time()

print ("Tempo Pi: %.8f s" %(toc-tic))
```

**TypeError: 'float' object
cannot be interpreted as
an integer**

Ok. Mas onde está a
origem do problema?

Pi paralelo

```
from multiprocessing import Process
import time
PROCS = 2
def pi(start, end, step):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    print(sum)
if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
    proc_size = num_steps // PROCS # Forçaremos uma divisão inteira

    tic = time.time()
    workers = []
    for i in range(PROCS):
        worker = Process(target=pi, args=(i*proc_size, (i+1)*proc_size - 1, step, ))
        workers.append(worker)
```

```
        for worker in workers :
            worker.start()
        for worker in workers :
            worker.join()
        toc = time.time()

        print ("Tempo Pi: %.8f s" %(toc-tic))
```

Pi paralelo

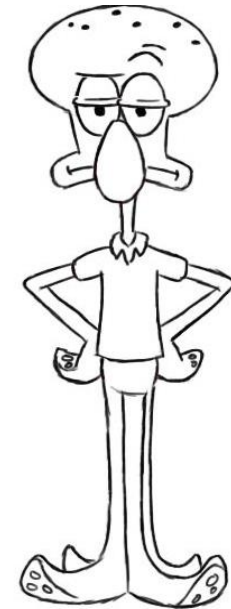
```
from multiprocessing import Process
import time
PROCS = 2
def pi(start, end, step):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    print(sum)
if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
    proc_size = num_steps // PROCS # Forçaremos uma divisão inteira

    tic = time.time()
    workers = []
    for i in range(PROCS):
        worker = Process(target=pi, args=(i*proc_size, (i+1)*proc_size - 1, step, ))
        workers.append(worker)
```

```
for worker in workers :
    worker.start()
for worker in workers :
    worker.join()
toc = time.time()

print ("Tempo Pi: %.8f s" %(toc-tic))
```

“Fácil”. Agora é só somar os resultados impressos na tela. Depois multiplicar por step.



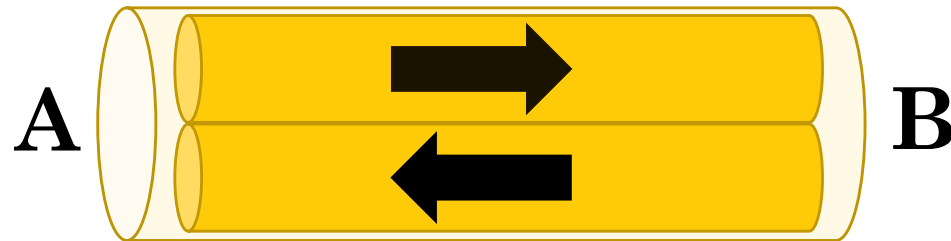
Trocando dados entre processos usando **PIPES**

Inter Process Communication (IPC) usando Pipes

- Pipe (Tubo)
 - Para comunicação entre dois processos
 - Um Pipe tem duas extremidades: o processo A escreve algo em sua extremidade do Pipe e o processo B pode lê-lo de sua
 - Pipes são bidirecionais

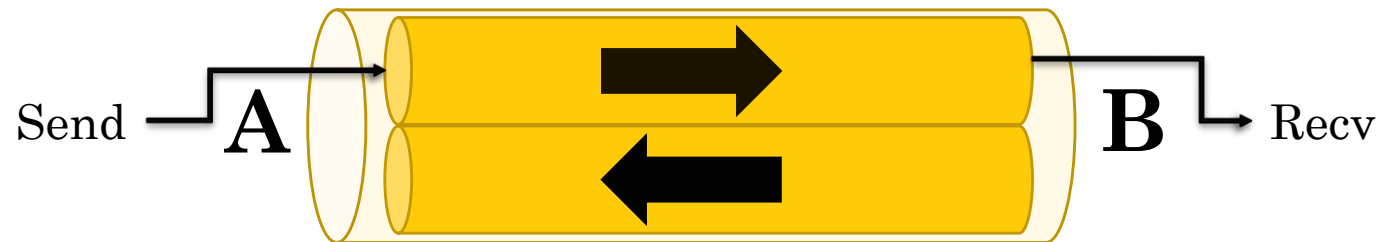
Pipes

- Um pipe tem duas extremidades: `a, b = Pipe()`
- Um processo envia algo para um lado e o outro processo pode receber do outro
- `recv` irá bloquear se o pipe estiver vazio



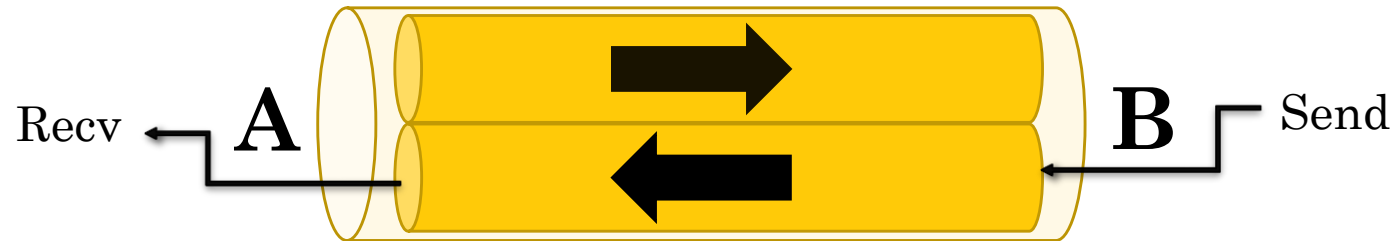
Pipes

- Um pipe tem duas extremidades: `a, b = Pipe()`
- Um processo envia algo para um lado e o outro processo pode receber do outro
- `recv` irá bloquear se o pipe estiver vazio



Pipes

- Um pipe tem duas extremidades: `a, b = Pipe()`
- Um processo envia algo para um lado e o outro processo pode receber do outro
- `recv` irá bloquear se o pipe estiver vazio



Inter-Process Communication (IPC) usando Pipes

- **Atenção:** Os dados de um pipe podem ficar corrompidos se dois processos (ou threads) tentarem ler ou gravar **na mesma extremidade** do pipe ao mesmo tempo.
- Naturalmente, não há risco de corrupção de processos usando diferentes extremidades do tubo ao mesmo tempo.
- **Atenção:** Se um processo for eliminado enquanto estiver tentando ler ou gravar em um pipe, é provável que os dados no pipe se tornem corrompidos, porque pode ser impossível ter certeza de onde estão os limites de mensagens.

Inter-Process Communication (IPC) usando Pipes

- `send(obj)`
 - Envie um objeto para a outra extremidade da conexão, que deve ser lida usando `recv()`.
- `recv()`
 - Retorna um objeto enviado da outra extremidade da conexão usando `send()`.
 - Bloqueia até que haja algo para receber.

Inter-Process Communication (IPC) usando Pipes

- `send(obj)`
 - Envie um objeto para a outra extremidade da conexão, que deve ser lida usando `recv()`.
 - O objeto deve ser particionável. Partições muito grandes (aproximadamente +32 MB, embora dependa do sistema operacional) podem gerar uma exceção `ValueError`.
- `recv()`
 - Retorna um objeto enviado da outra extremidade da conexão usando `send()`.
 - Bloqueia até que haja algo para receber.
 - Gera a exceção `EOFError` se não houver mais nada para receber e o outro lado estiver fechado.

Exemplo 3: comunicador PIPE

```
from multiprocessing import Process , Pipe

def worker(connA) :
    return

def master(connB) :
    return

if __name__ == '__main__' :
    a, b = Pipe()
    w = Process(target = worker, args = (a, ) )
    m = Process(target = master, args = (b, ) )
    w.start()
    m.start()
    w.join()
    m.join()
```

Exemplo 3: comunicador PIPE

```
from multiprocessing import Process , Pipe
```

```
def worker(conn) :  
    while True :  
        item = conn.recv()  
        if item == 'fim' :  
            break  
        print(item)
```

```
def master(conn) :  
    conn.send(' Isto')  
    conn.send(' esta')  
    conn.send(' funcionando?')  
    conn.send('fim')
```

Saída:
Isto
esta
funcionando?

```
if __name__ == '__main__' :  
    a, b = Pipe()  
    w = Process(target = worker, args = (a, ) )  
    m = Process(target = master, args = (b, ) )  
    w.start()  
    m.start()  
    w.join()  
    m.join()
```


Exemplo 3: comunicador PIPE

```
from multiprocessing import Process , Pipe
```

```
def worker(conn) :
```

```
    while True :
```

```
        item = conn.recv()
```

```
        if item == 'fim' :
```

```
            break
```

```
        print(item)
```

```
def master(conn) :
```

```
    conn.send(' Isto')
```

```
    conn.send(' esta')
```

```
    conn.send(' funcionando?')
```

```
    conn.send('fim')
```

```
if __name__ == '__main__' :
```

```
    a, b = Pipe()
```

```
    w = Process(target = worker, args = (a, ) )
```

```
    m = Process(target = master, args = (b, ) )
```

```
    w.start()
```

```
    m.start()
```

```
    w.join()
```

```
    m.join()
```

Exemplo 3: comunicador PIPE

```
from multiprocessing import Process , Pipe
```

```
def worker(conn) :
```

```
    while True :
```

```
        item = conn.recv()
```

```
        if item == 'fim' :
```

```
            break
```

```
        print(item)
```

```
def master(conn) :
```

```
    conn.send(' Isto')
```

```
    conn.send(' esta')
```

```
    conn.send(' funcionando?')
```

```
    conn.send('fim')
```

Será que o trabalhador
pode falar com o mestre?



```
if __name__ == '__main__' :
```

```
    a, b = Pipe()
```

```
    w = Process(target = worker, args = (a, ) )
```

```
    m = Process(target = master, args = (b, ) )
```

```
    w.start()
```

```
    m.start()
```

```
    w.join()
```

```
    m.join()
```

Exemplo 3: comunicador PIPE

```
from multiprocessing import Process , Pipe
```

```
def worker(conn) :
```

```
    while True :
```

```
        item = conn.recv()
```

```
        if item == 'fim' :
```

```
            break
```

```
        print item
```

```
        conn.send('Claro!')
```

```
def master(conn) :
```

```
    conn.send(' Isto')
```

```
    conn.send(' esta')
```

```
    conn.send(' funcionando?')
```

```
    conn.send('fim')
```

```
    item = conn.recv()
```

```
    print item
```

Saída:

Isto

esta

funcionando?

Claro!

```
if __name__ == '__main__' :
```

```
    a, b = Pipe()
```

```
    w = Process(target = worker, args = (a, ) )
```

```
    m = Process(target = master, args = (b, ) )
```

```
    w.start()
```

```
    m.start()
```

```
    w.join()
```

```
    m.join()
```

Exercício 2

- Transforme o programa Pi em paralelo utilizando PIPES.
- **Vamos precisar:**
 1. Criar um pipe e atribuir a duas variáveis
 2. Enviar uma ponta do pipe para todos os processos
 3. Os processos filho irão chamar `send()`
 4. O processo master irá chamar `recv()`

Pi paralelo + PIPE

```
from multiprocessing import Process, Pipe
import time

PROCS = 2
a,b = Pipe()

def pi(start, end, step, sums):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    sums.send(sum)

if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
    proc_size = num_steps // PROCS
```

```
tic = time.time()
workers = []
for i in range(PROCS):
    worker = Process( target=pi , args =(i*proc_size,
(i+1)*proc_size - 1, step, a))
    workers.append(worker)

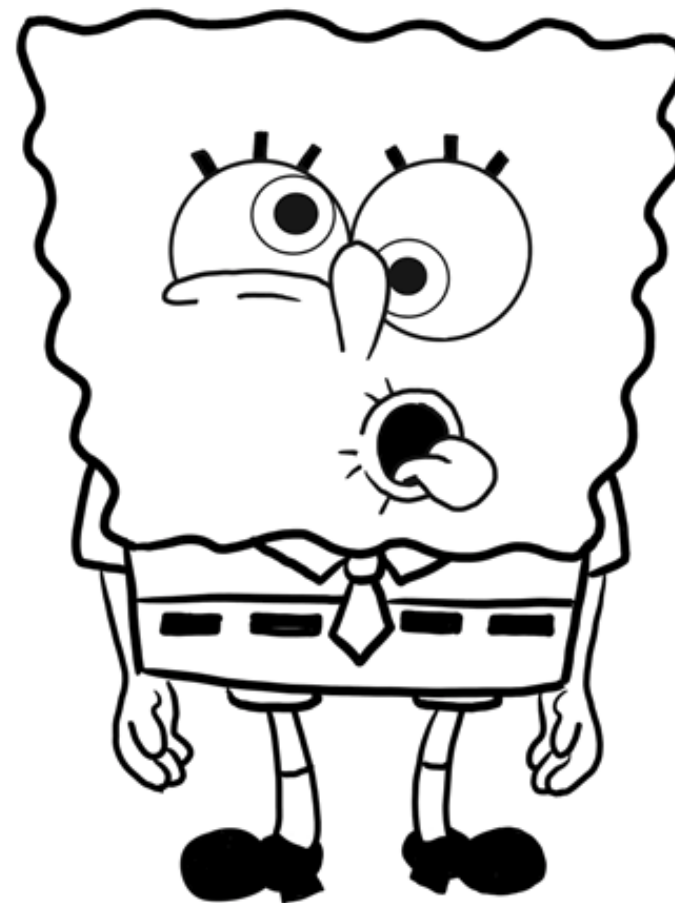
for worker in workers :
    worker.start()
for worker in workers :
    worker.join()
toc = time.time()

for i in range(PROCS):
    sum += b.recv()
pi = step * sum
print ("Valor Pi: %.20f" %pi)
print ("Tempo Pi: %.8f s" %(toc-tic))
```

Lembrando

- **Atenção:** Os dados de um pipe podem ficar corrompidos se dois processos tentarem ler ou gravar na mesma extremidade do pipe ao mesmo tempo.

Ops. Exatamente o
que estamos
tentando fazer!



Pi paralelo + PIPE

```
from multiprocessing import Process, Pipe
import time

PROCS = 2
a,b = Pipe()

def pi(start, end, step, sums):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.
```

```
    sums.send(sum)
```

Condição de corrida!!!

```
if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
    proc_size = num_steps // PROCS
```

```
tic = time.time()
workers = []
for i in range(PROCS):
    worker = Process( target=pi , args =(i*proc_size,
(i+1)*proc_size - 1, step, a))
    workers.append(worker)

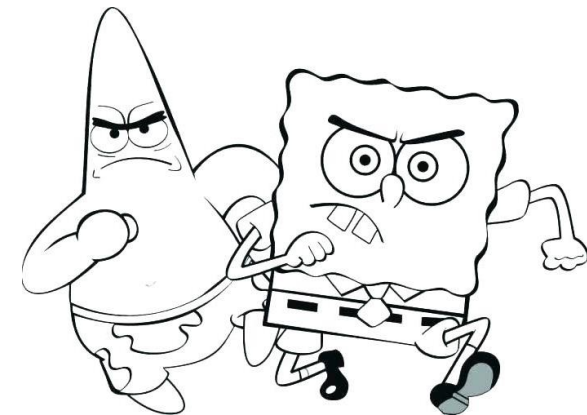
for worker in workers :
    worker.start()
for worker in workers :
    worker.join()
```

```
for i in range(PROCS):
    sum += b.recv()
pi = step * sum
print ("Valor Pi: %.20f" %pi)
print ("Tempo Pi: %.8f s" %(toc-tic))
```

Sincronizando regiões críticas

Sincronização e Regiões críticas

- A principal causa da ocorrência de erro na programação de processos está relacionada com o fato do uso de estruturas de dados compartilhadas.
- O problema existe quando dois ou mais processos tentam acessar/alterar as mesmas estruturas de dados (race conditions).



Exemplo clássico

Tempo	P1	P2	Saldo
T0			\$200

Exemplo clássico

Tempo	P1	P2	Saldo
T0			\$200
T1	Leia Saldo \$200		\$200
T2		Leia Saldo \$200	\$200

Exemplo clássico

Tempo	P1	P2	Saldo
T0			\$200
T1	Leia Saldo \$200		\$200
T2		Leia Saldo \$200	\$200
T3		Some \$100 \$300	\$200
T4	Some \$150 \$350		\$200

Exemplo clássico

Tempo	P1	P2	Saldo
T0			\$200
T1	Leia Saldo \$200		\$200
T2		Leia Saldo \$200	\$200
T3		Some \$100 \$300	\$200
T4	Some \$150 \$350		\$200
T5		Escreva Saldo \$300	\$300
T6	Escreva Saldo \$350		\$350

Exemplo clássico

Tempo	P1	P2	Saldo
T0	Devemos garantir que não importa a ordem de execução (escalonamento), teremos sempre um resultado consistente!		
T1			
T2			
T3			
T4			
T5		Escreva Saldo \$300	\$300
T6	Escreva Saldo \$350		\$350

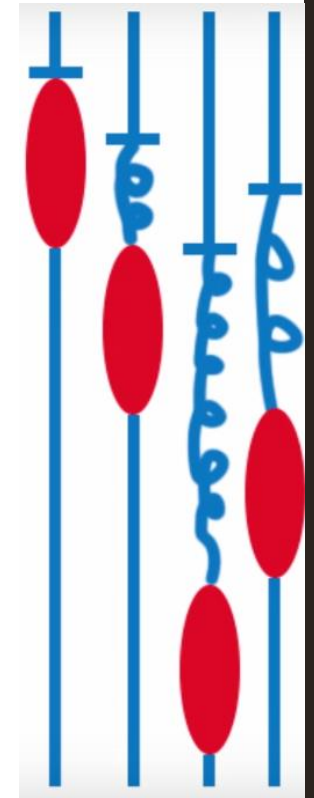
Expandindo o exemplo

- Agora imagine que estamos tentando fazer um saque da conta.
- No mesmo tempo, eu estou tentando ler o saldo para prosseguir com o pagamento de uma conta.
- Note que, **durante uma atualização** de variável, **não é seguro fazer leituras** dessa variável, pois ela não estará estável.
- **Ou seja, durante uma escrita, não devemos fazer outra escrita/leitura sem garantir exclusividade.**

Controlando acesso a recursos usando **LOCKS**

locks

- Em situações em que um único recurso precisa ser compartilhado entre vários processos, um bloqueio pode ser usado para evitar acessos conflitantes.
- `acquire(block=True, timeout=None)`
 - Adquira um Lock, bloqueando ou não bloqueando.
 - Se `block = True` (padrão), a chamada será bloqueada até adquirir o lock.
 - Se `block = False`, a chamada do método não é bloqueada.
 - O `timeout` define o tempo limite de bloqueio em segundos, caso o lock não possa ser adquirido. Valores negativos definem o tempo como zero. Se nenhum valor for definido (o padrão) o tempo limite será infinito.
- `release()`
 - Solte um Lock. Isso pode ser chamado de qualquer processo ou thread, não apenas o processo ou thread que originalmente adquiriu o bloqueio.



Controlando acesso a recursos

```
import multiprocessing
import sys

if __name__ == "__main__":
    w1 = multiprocessing.Process(target=trabA, args=(sys.stdout, ))
    w2 = multiprocessing.Process(target=trabB, args=(sys.stdout, ))
    w1.start()
    w2.start()
    w1.join()
    w2.join()
```

Controlando acesso a recursos

```
import multiprocessing
import sys
def trabA(stream):
    stream.write('Trabalhador 1 imprimindo\n')

def trabB(stream):
    stream.write('Trabalhador 2 imprimindo\n')

if __name__ == "__main__":
    w1 = multiprocessing.Process(target=trabA, args=(sys.stdout, ))
    w2 = multiprocessing.Process(target=trabB, args=(sys.stdout, ))
    w1.start()
    w2.start()
    w1.join()
    w2.join()
```

Neste exemplo, as mensagens impressas na tela podem sair misturadas se os dois processos não sincronizarem o acesso ao fluxo de saída.

Saída:
Trabalhador 1 Trabalhador imprimindo
2 imprimindo

Controlando acesso a recursos

```
import multiprocessing
import sys
def trabA(lock, stream):
    with lock:
        stream.write('Trabalhador 1 imprimindo\n')

def trabB(lock, stream):
    lock.acquire()
    stream.write('Trabalhador 2 imprimindo\n')
    lock.release()

if __name__ == "__main__":
    lock = multiprocessing.Lock()
    w1 = multiprocessing.Process(target=trabA, args=(lock, sys.stdout, ))
    w2 = multiprocessing.Process(target=trabB, args=(lock, sys.stdout, ))
    w1.start()
    w2.start()
    w1.join()
    w2.join()
```

Duas maneiras de fazer a mesma ação

Saída:
Trabalhador 1 imprimindo
Trabalhador 2 imprimindo

Exercício

- Transforme o programa Pi em paralelo usando PIPES com locks!
- Agora vamos utilizar locks para que nosso programa seja multiprocess-safe
- **Vamos precisar:**
 1. Criar um lock
 2. Enviar o lock como parâmetro
 3. Antes de inserir no pipe, vamos usar o lock..... “**with lock:**”

Pi paralelo com PIPE+lock

```
from multiprocessing import Process, Pipe, Lock
import time

PROCS = 2
a,b = Pipe()

def pi(lk, start, end, step, sums):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    with lk:
        sums.send(sum)

if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
```

```
proc_size = num_steps // PROCS
tic = time.time()
workers = []
lock = Lock()
for i in range(PROCS):
    worker = Process( target=pi , args =(lock,
i*proc_size, (i+1)*proc_size - 1, step, a))
    workers.append(worker)

for worker in workers :
    worker.start()
for worker in workers :
    worker.join()
toc = time.time()

for i in range(PROCS):
    sum += b.recv()
pi = step * sum
print ("Valor Pi: %.20f" %pi)
print ("Tempo Pi: %.8f s" %(toc-tic))
```

Pi paralelo com PIPE+lock

```
from multiprocessing import Process, Pipe, Lock
import time

PROCS = 2
a,b = Pipe()

def pi(lk, start, end, step, sums):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    with lk:
        sums.send(sum)

if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
```

```
proc_size = num_steps // PROCS
tic = time.time()
world = []
for i in range(0, num_steps, proc_size):
    p = Process(target=pi, args=(lock,
                                i*proc_size, (i+1)*proc_size, step, a))
    world.append(p)
    p.start()
for worker in world:
    worker.join()
toc = time.time()

for i in range(PROCS):
    sum += b.recv()
pi = step * sum
print ("Valor Pi: %.20f" %pi)
print ("Tempo Pi: %.8f s" %(toc-tic))
```

Não será
necessário
lock no
recv()

Afinal apenas
um processo
irá chamar o
recv()

Pi paralelo com PIPE+lock

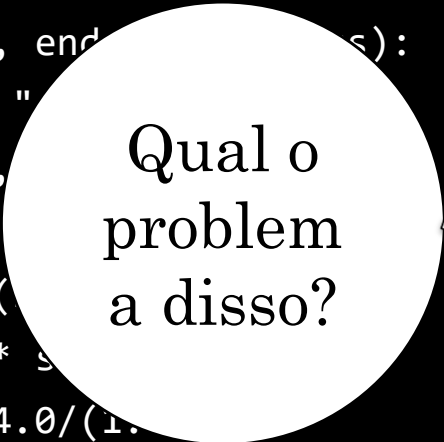
```
from multiprocessing import Process, Pipe, Lock
import time

PROCS = 2
a,b = Pipe()
```

```
def pi(lk, start, end):
    print ("Start: ", start)
    print ("End: ", end)
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x*x*x)
    with lk:
        sums.send(sum)
```

```
if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
```

Qual o
problem
a disso?



```
proc_size = num_steps // PROCS
tic = time.time()
workers = []
lock = Lock()
for i in range(PROCS):
    worker = Process( target=pi , args =(lock,
i*proc_size, (i+1)*proc_size - 1, step, a))
    workers.append(worker)

for worker in workers :
    worker.start()
    sum += b.recv()
for worker in workers :
    worker.join()
toc = time.time()

for i in range(PROCS):
sum += b.recv()
pi = step * sum
print ("Valor Pi: %.20f" %pi)
print ("Tempo Pi: %.8f s" %(toc-tic))
```



Pi paralelo com PIPE+lock

```
from multiprocessing import Process, Pipe, Lock
import time
PROCS = 2
a,b = Pipe()
```

```
def pi(lk, start, step):
    print ("Start")
    print ("Step: ", step)
    sum = 0
    for i in range(start, start+step):
        x = 1/(i+1)
        sum += x
    with lk:
        sums.append(sum)
```

```
if __name__ == '__main__':
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
```

O método `recv()` é bloqueante.
Ou seja, o próximo processo só será iniciado quando o anterior terminar

Também conhecido como execução sequencial

```
proc_size = num_steps // PROCS
tic = time.time()
workers = []
lock = Lock()
for i in range(PROCS):
    worker = Process(target=pi, args=(lock, i*proc_size, (i+1)*proc_size - 1, step, a))
    workers.append(worker)

for worker in workers:
    worker.start()
    sum += b.recv()
for worker in workers:
    worker.join()
toc = time.time()

for i in range(PROCS):
    sum += b.recv()
pi = step * sum
print ("Valor Pi: %.20f" %pi)
print ("Tempo Pi: %.8f s" %(toc-tic))
```

Pi paralelo com PIPE+lock

```
from multiprocessing import Process, Pipe, Lock
import time

PROCS = 2
a,b = Pipe()

def pi(lk, start, end, step, sums):
    print ("Start: ", str(start))
    print ("End: ", str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        sum = sum + 4.0/(1.0+x*x)
    with lk:
        sums.send(sum)

if __name__ == "__main__":
    num_steps = 100_000_000
    sum = 0.0
    step = 1.0/num_steps
```

```
proc_size = num_steps // PROCS
tic = time.time()
workers = []
lock = Lock()
for i in range(PROCS):
    worker = Process( target=pi , args =(lock,
i*proc_size, (i+1)*proc_size - 1, step, a))
    workers.append(worker)

for worker in workers :
    worker.start()
for worker in workers :
    worker.join()
toc = time.time()

for i in range(PROCS):
    sum += b.recv()
pi = step * sum
print ("Valor Pi: %.20f" %pi)
print ("Tempo Pi: %.8f s" %(toc-tic))
```

Value e array compartilhados

Value

- `Value(typecode, value)`
- Crie um **objeto compartilhável** com um atributo de valor gravável e retorne um ponteiro para ele.
- **Retorna um objeto ctypes alocado na memória compartilhada.** O valor em si pode ser acessado através do atributo `value`.
- **`typecode_or_type (ctypes)`** determina o tipo do objeto retornado.

Type Codes → C_Type

Type code	C Type	Python Type	Minimum size in bytes	Notes
'b'	signed char	int	1	
'B'	unsigned char	int	1	
'u'	Py_UNICODE	Unicode character	2	(1)
'h'	signed short	int	2	
'H'	unsigned short	int	2	
'i'	signed int	int	2	
'I'	unsigned int	int	2	
'l'	signed long	int	4	
'L'	unsigned long	int	4	
'q'	signed long long	int	8	(2)
'Q'	unsigned long long	int	8	(2)
'f'	float	float	4	
'd'	double	float	8	

Exemplo com value

```
import time
from multiprocessing import Process, Value

if __name__ == '__main__':
    v = Value('i', 0)
    procs = [Process(target=func, args=(v,)) for i in range(10)]

    for p in procs: p.start()
    for p in procs: p.join()

    print (v.value)
```

Exemplo com value

```
import time
from multiprocessing import Process, Value

def func(val):
    for i in range(50):
        time.sleep(0.01)
        val.value += 1

if __name__ == '__main__':
    v = Value('i', 0)
    procs = [Process(target=func, args=(v,)) for i in range(10)]

    for p in procs: p.start()
    for p in procs: p.join()

    print (v.value)
```

Funciona?

Exemplo com value

```
import time
from multiprocessing import Process, Value
```

```
def func(val):
    for i in range(50):
        time.sleep(0.01)
```

```
    val.value += 1
```

Temos uma nova condição de corrida!!!

```
if __name__ == '__main__':
```

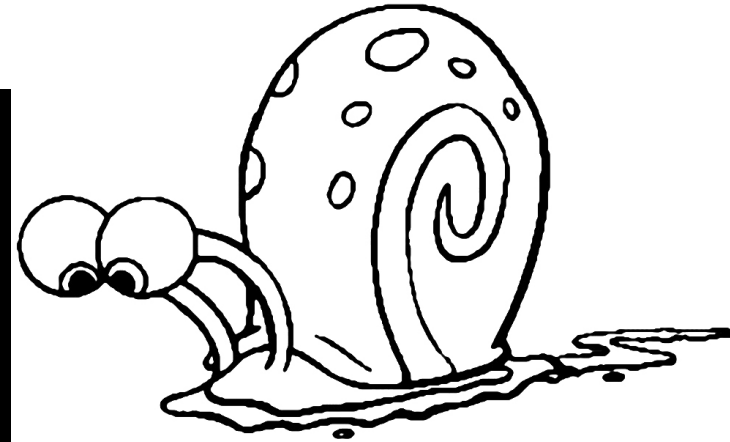
```
    v = Value('i', 0)
```

```
    procs = [Process(target=func, args=(v,)) for i in range(10)]
```

```
    for p in procs: p.start()
```

```
    for p in procs: p.join()
```

```
    print (v.value)
```



Value

- `Value(typecode, value, lock=True)`
- Se lock for True (o padrão), um novo objeto de bloqueio será criado para sincronizar o acesso ao valor.
- Operações como `+=` que envolvem leitura e gravação não são atômicas. Então, se, por exemplo, você quiser incrementar atomicamente um valor compartilhado, é insuficiente fazer apenas

```
contador.valor += 1
```

- Mas, você pode fazer:

```
with counter.get_lock():  
    contador.valor + = 1
```

Exemplo com value

```
import time
from multiprocessing import Process, Value

def func(val):
    for i in range(50):
        time.sleep(0.01)
        val.value += 1

if __name__ == '__main__':
    v = Value('i', 0, lock=True)
    procs = [Process(target=func, args=(v,)) for i in range(10)]

    for p in procs: p.start()
    for p in procs: p.join()

    print (v.value)
```

Qual o problema?

Exemplo com value

```
import time
from multiprocessing import Process, Value

def func(val):
    for i in range(50):
        time.sleep(0.01)
        val.value += 1

if __name__ == '__main__':
    v = Value('i', 0, lock=True)
    procs = [Process(target=func, args=(v,)) for i in range(10)]

    for p in procs: p.start()
    for p in procs: p.join()

    print (v.value)
```

Definir um lock para um VALUE
**não significa que ele usará
automaticamente esse lock!!!**

Exemplo com value

```
import time
from multiprocessing import Process, Value

def func(val):
    for i in range(50):
        time.sleep(0.01)
        with val.get_lock():
            val.value += 1

if __name__ == '__main__':
    v = Value('i', 0, lock=True)
    procs = [Process(target=func, args=(v,)) for i in range(10)]

    for p in procs: p.start()
    for p in procs: p.join()

    print (v.value)
```

Yes! Agora temos uma
função confiável

Array

- `Array(typecode, sequence)`
- Crie uma matriz e retorne um proxy para ela.
- Retorna uma matriz ctypes alocada da memória compartilhada.
- **typecode_or_type** determina o tipo dos elementos da matriz retornada.

Array

- `Array(typecode, sequence)`
- Se o bloqueio for `True` (o padrão), um novo objeto de bloqueio será criado para sincronizar o acesso ao valor.
- Observe que uma matriz de `ctypes.c_char` tem valor e atributos brutos que permitem usá-lo para armazenar e recuperar strings.

Type Codes

Type code	C Type	Python Type	Minimum size in bytes	Notes
'b'	signed char	int	1	
'B'	unsigned char	int	1	
'u'	Py_UNICODE	Unicode character	2	(1)
'h'	signed short	int	2	
'H'	unsigned short	int	2	
'i'	signed int	int	2	
'I'	unsigned int	int	2	
'l'	signed long	int	4	
'L'	unsigned long	int	4	
'q'	signed long long	int	8	(2)
'Q'	unsigned long long	int	8	(2)
'f'	float	float	4	
'd'	double	float	8	

Exercício

- Transforme o programa Pi em paralelo usando (VALUE ou ARRAY)!
- **Vamos precisar:**
 1. Criar uma variável Value, e envia-la a todos os processos filho.
 2. Cada processo filho calcula sua parcela.
 3. Ao final do cálculo, cada filho obtém o lock e atualiza a variável compartilhada.

Pi paralelo usando Value

```
from multiprocessing import Process, Value
import time

PROCS = 2

def pi(val, start, end, step):
    print("Start: " + str(start))
    print("End: " + str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        with val.get_lock():
            val.value += 4.0/(1.0+x*x)

if __name__ == "__main__":
    num_steps = 100000000
    step = 1.0/num_steps
    proc_size = num_steps // PROCS
```

Problemas?

```
tic = time.time()
workers = []
v = Value('d', 0, lock=True)
for i in range(PROCS):
    worker = Process(target=pi, args=(v,
i*proc_size, (i+1)*proc_size-1, step))
    workers.append(worker)

for worker in workers :
    worker.start()
for worker in workers :
    worker.join()
toc = time.time()

pi = step * v.value
print("Valor Pi: %.20f" %pi)
print("Tempo Pi: %.8f s" %(toc-tic))
```

Pi paralelo usando Value

```
from multiprocessing import Process, Value
import time

PROCS = 2

def pi(val, start, end, step):
    print("Start: " + str(start))
    print("End: " + str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        with val.get_lock():
            val.value += 4.0/(1.0+x*x)

if __name__ == "__main__":
    num_steps = 100000000
    step = 1.0/num_steps
    proc_size = num_steps // PROCS
```

N chamadas
get_lock()

Importante: Devemos
evitar ao máximo
chamar o lock!

O excesso de chamadas
get_lock() irá derrubar
o desempenho

```
tic = time.time()
workers = []
v = Value('d', 0, lock=True)

for i in range(0, num_steps, proc_size):
    p = Process(target=pi, args=(v, i, i+proc_size, step))
    workers.append(p)
    p.start()

for p in workers:
    p.join()

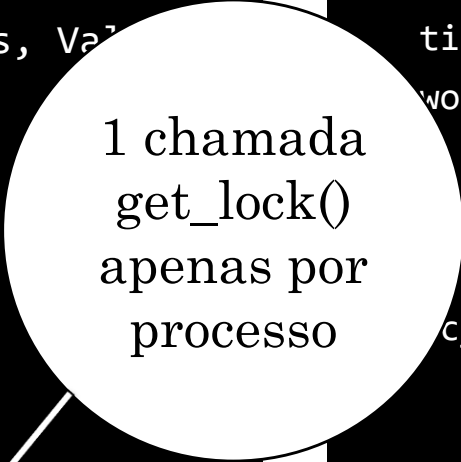
toc = time.time()
print("Resultado de %f" %pi)
print("Tempo de execução: %.8f s" %(toc-tic))
```

Pi paralelo usando Value

```
from multiprocessing import Process, Value
import time
PROCS = 2
def pi(val, start, end, step):
    print("Start: " + str(start))
    print("End: " + str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        with val.get_lock():
            val.value += 4.0/(1.0+x*x)

if __name__ == "__main__":
    num_steps = 100000000
    step = 1.0/num_steps
    proc_size = num_steps // PROCS
```

1 chamada
get_lock()
apenas por
processo



```
tic = time.time()
workers = []
v = Value('d', 0, lock=True)
for i in range(PROCS):
    worker = Process(target=pi, args=(v,
proc_size, (i+1)*proc_size-1, step))
    workers.append(worker)

for worker in workers:
    worker.start()
for worker in workers:
    worker.join()
toc = time.time()

pi = step * v.value
print("Valor Pi: %.20f" %pi)
print("Tempo Pi: %.8f s" %(toc-tic))
```

Pi paralelo usando Value

```
from multiprocessing import Process, Value
import time

PROCS = 2

def pi(val, start, end, step):
    print("Start: " + str(start))
    print("End: " + str(end))
    sum = 0.0
    for i in range(start, end):
        x = (i+0.5) * step
        with val.get_lock():
            val.value += 4.0/(1.0+x*x)

if __name__ == "__main__":
    num_steps = 100000000
    step = 1.0/num_steps
    proc_size = num_steps // PROCS
```

```
tic = time.time()
workers = []
v = Value('d', 0, lock=True)
for i in range(PROCS):
    worker = Process(target=pi, args=(v,
i*proc_size, (i+1)*proc_size-1, step))
    workers.append(worker)

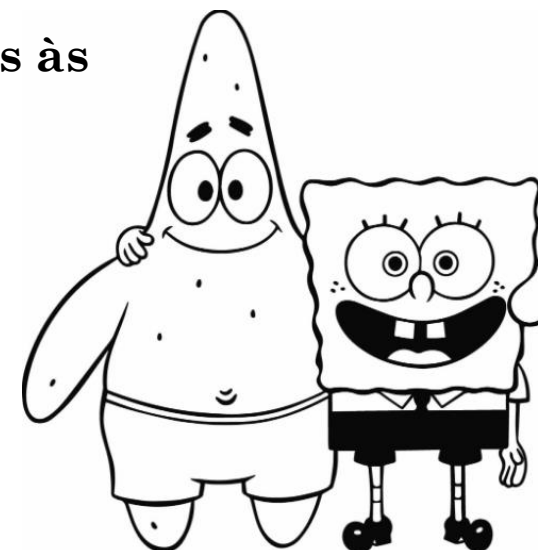
for worker in workers :
    worker.start()
for worker in workers :
    worker.join()
toc = time.time()

pi = step * v.value
print("Valor Pi: %.20f" %pi)
print("Tempo Pi: %.8f s" %(toc-tic))
```

Pool de procesos

A classe Pool

- Uma abordagem mais **conveniente e amigável** para tarefas simples de processamento paralelo é fornecida pela classe Pool.
 - Quatro métodos são particularmente interessantes:
 - Pool.apply
 - Pool.map
 - Pool.apply_async
 - Pool.map_async
- } **Assíncronas**
- Os métodos Pool.apply e Pool.map são basicamente **equivalentes às funções** de aplicar e mapear do Python.



Pool.apply

Pool.apply

- **Antigamente**, no Python, para chamar uma função com argumentos arbitrários, você usaria `apply`:

`apply(f, args, kwargs)`

- **Pool.apply** é como Python `apply`.
 - Porém, a chamada de função é realizada em um **processo separado**.
 - `Pool.apply` **bloqueia** até que a função seja concluída.

Pool.apply

- **Antigamente**, no Python, para chamar uma função com argumentos arbitrários, você usaria `apply`:

`apply(f, args, kwargs)`

- **Pool.apply** é como Python `apply`.
 - Porém, a chamada de função é realizada em um **processo separado**.
 - `Pool.apply` **bloqueia** até que a função seja concluída.
- **Pool.apply_async** retorna imediatamente em vez de esperar pelo resultado.
 - Um objeto `AsyncResult` é retornado.
 - Você chama seu método `get()` para recuperar o resultado.
 - O método `get()` bloqueia até que a função seja concluída.

Múltiplos processos com `apply()`

- O método **`apply()` paralelo** não irá subdividir os dados de entrada.
- Assim, podemos mapear dados distintos e até funções distintas.

```
from multiprocessing import Pool
def func(val):
    return val*val
if __name__ == '__main__':
    itens = [1, 2, 3]
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!
    resultado = []
    for i in itens:
        resultado.append(p.apply(func, [i])) # Irá bloquear até terminar!
    print(resultado)
    p.close()
    p.join()
```

Múltiplos processos com `apply()`

- O método `apply()` não irá subdividir os dados de entrada.
- Assim, podemos mapear dados distintos e até fun

```
from multiprocessing import Pool
def func(val):
    return val*val
if __name__ == '__main__':
    itens = [1, 2, 3]
    p = Pool(8) # Cria um conjunto de 8 trab
    resultado = []
    for i in itens:
        resultado.append(p.apply(func, [i]))
    print(resultado)
    p.close()
    p.join()
```

```
# Irá bloquear até terminar!
# Irá bloquear até terminar!
# Irá bloquear até terminar!
# Irá bloquear até terminar!
# Irá bloquear até terminar!
# Irá bloquear até terminar!
# Irá bloquear até terminar!
# Irá bloquear até terminar!
# Irá bloquear até terminar!
# Irá bloquear até terminar!
```

Múltiplos processos com `apply_async()`

- `Pool.apply_async` retorna imediatamente sem esperar o resultado.
- Devemos chamar `get()` para recuperar o resultado.
- O método `get()` é bloqueado até que a função seja concluída.

```
if __name__ == '__main__':  
    itens = [1, 2, 3]  
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!  
    resultado = []  
    for i in itens:  
        resultado.append(p.apply_async(func, [i])) # Não irá bloquear!  
    output = [p.get() for p in resultado]  
    print(output)
```

Usando `apply()` de outras formas...

```
if __name__ == '__main__':
    itens = [1, 2, 3]
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!
    resultado = []
    for i in itens:
        resultado.append(p.apply(func, [i])) # Irá bloquear até terminar!
    print(resultado)
```

```
if __name__ == '__main__':
    itens = [1, 2, 3]
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!
    resultado = [p.apply(func, args=(i,)) for i in itens]
    print(resultado)
```

```
if __name__ == '__main__':
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!
    resultado = [p.apply(func, args=(i,)) for i in range(1,4)]
    print(resultado)
```

Usando `apply()` de outras formas...

```
if __name__ == '__main__':  
    itens = [1, 2, 3]  
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!  
    resultado = []  
    for i in itens:  
        resultado.append(p.apply(func, [i])) # Irá bloquear  
    print(resultado)
```

```
if __name__ == '__main__':  
    itens = [1, 2, 3]  
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!  
    resultado = [p.apply(func, args=(i,)) for i in itens]  
    print(resultado)
```

```
if __name__ == '__main__':  
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!  
    resultado = [p.apply(func, args=(i,)) for i in range(1,4)]  
    print(resultado)
```

Qual o
problema?

Usando `apply()` de outras formas...

```
if __name__ == '__main__':  
    itens = [1, 2, 3]  
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!  
    resultado = []  
    for i in itens:  
        resultado.append(p.apply(func, [i])) # Irá bloquear  
    print(resultado)
```

```
if __name__ == '__main__':  
    itens = [1, 2, 3]  
    p = Pool(8) # Cria um conjunto  
    resultado = [p.apply(func, arg  
    print(resultado)
```

```
if __name__ == '__main__':  
    p = Pool(8) # Cria um conjunto  
    resultado = [p.apply(func, args=(  
    print(resultado)
```

Qual o
problema?

Devemos
usar a
versão
async!

Pool.map

A função map serial

- A **função serial** map() aplica um dado à função para cada item de um iterável e retorna uma lista dos resultados.

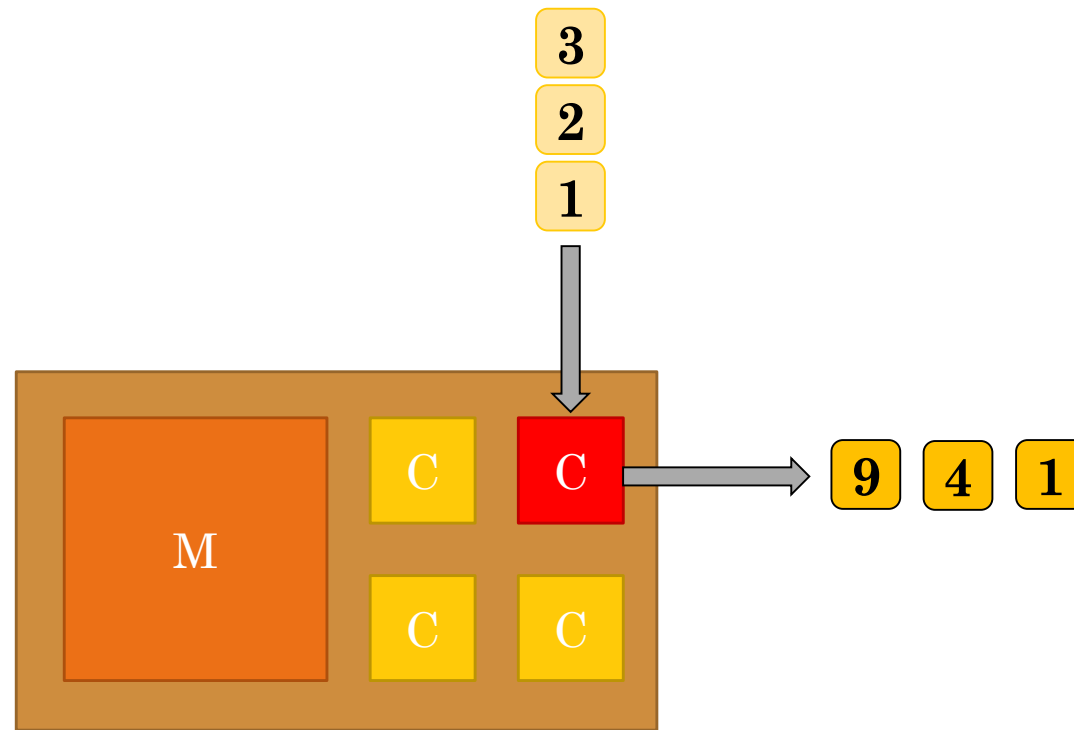
```
def func(val):  
    return val*val  
  
if __name__ == '__main__':  
    itens = [1, 2, 3]  
    resultado = map(func, itens) # mapeia os itens para a função  
    print(resultado)  
    numbersSquare = list(resultado) # converte objetos map -> list  
    print(numbersSquare)
```

Saída:

<map object at 0x7f00e775ce10>
[1, 4, 9]

Mapeando uma itens para uma função

- A **função serial** `map()` aplica um dado à função para cada item de um iterável e retorna uma lista dos resultados.



Múltiplos processos com `map()`

- A classe `Pool` representa um conjunto de processos de trabalho.
- Ele possui métodos que permitem que as tarefas sejam transferidas para os processos de trabalho de algumas maneiras diferentes.

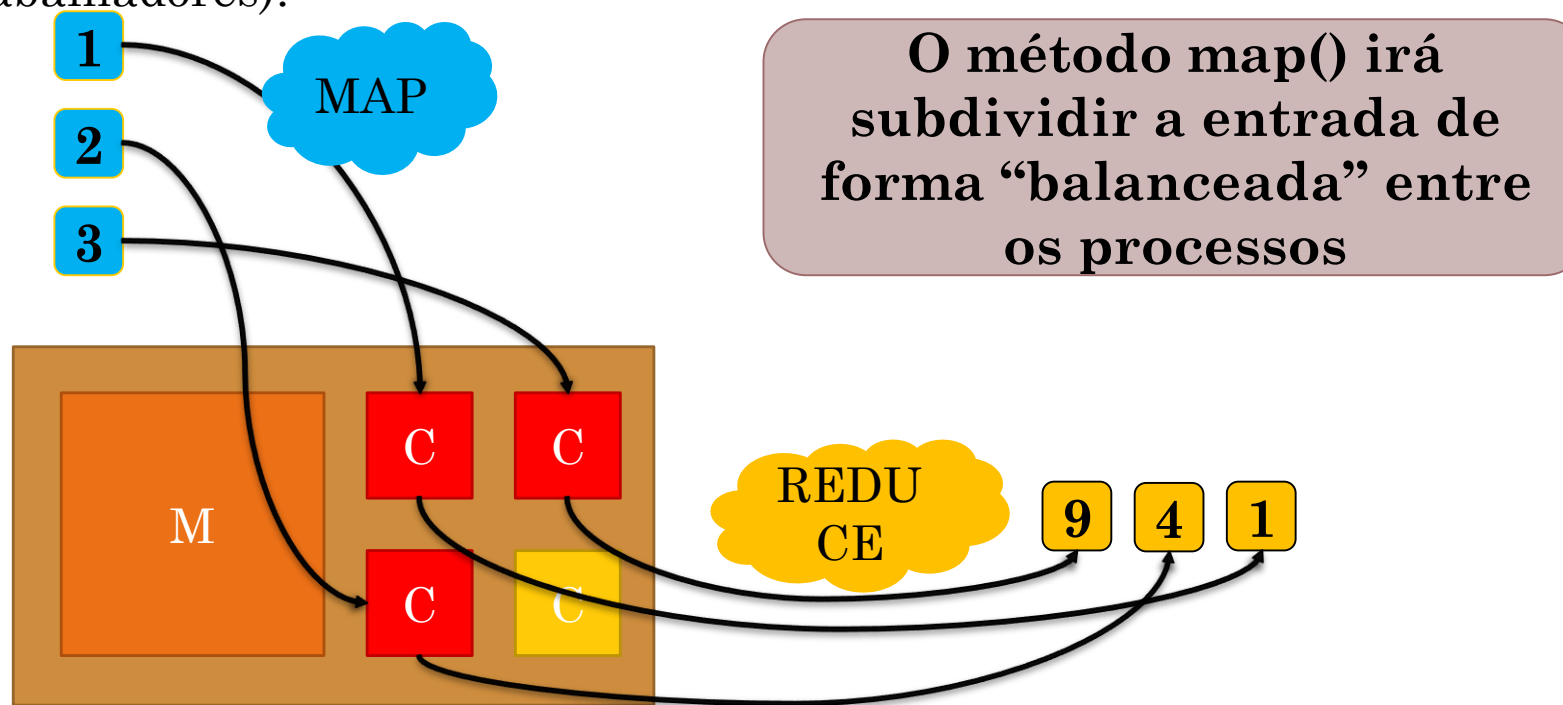
```
from multiprocessing import Pool

def func(val):
    return val*val

if __name__ == '__main__':
    itens = [1, 2, 3]
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!
    resultado = p.map(func, itens) # Irá bloquear até o término
    print(resultado)
    p.close()
    p.join()
```

Mapeando itens para uma função (em múltiplos processos)

- A função **map()** **paralela** aplica um dado à função para cada item de um iterável e retorna uma lista dos resultados.
- Tudo isso distribuindo entre os processos do Pool (cjto. de trabalhadores)!



Usando `map()` de outras formas...

```
if __name__ == '__main__':  
    itens = [1, 2, 3]  
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!  
    resultado = p.map(func, itens)  
    print(resultado)
```

```
if __name__ == '__main__':  
    p = Pool(8) # Cria um conjunto de 8 trabalhadores!  
    resultado = p.map(func, range(1,4))  
    print(resultado)
```

Argumentos

- `map(func, iterable[, chunksize])`
- Esse método divide o iterável em vários pedaços que ele envia ao pool de processos como tarefas separadas.
- O tamanho (aproximado) desses pedaços pode ser especificado definindo `chunksize` como um número inteiro positivo.

Cuidados com listas extensas

- `map(func, iterable[, chunksize])`
- Observe que **isso pode causar alto uso de memória** para iteráveis muito longos.
- **Considere usar `imap()`** ou `imap_unordered()` com opção de tamanho de bloco explícito para melhor eficiência.

Imap

Veja também
variantes
como o
starmap

- **imap()** é a versão preguiçosa de **map()**.
- **As tarefas são enviadas pouco a pouco (não requer alocação total de memória no principio)**
- Você pode trabalhar sobre os resultados, mesmo que nem todos os cálculos tenham terminado.

Apply vs. Map

- **Pool.apply**: quando você precisa executar uma função em outro processo.

```
p.apply(f, (0.1,))
```

- **Pool.map**: roda a função sobre um conjunto de argumentos em paralelo.

```
p.map(f, [0.3, 0.1, 0.2])
```

- As variantes assíncronas retornam uma promessa do resultado.

Exercício FINAL

- Transforme o programa Pi em paralelo usando POOL!

Python com numba

O que é o numba?

- O Numba é uma biblioteca do Python
- Permite acelerar o desempenho de funções em Python usando a compilação just-in-time (JIT).
- Ele pode ser usado para gerar código nativo de CPU a partir do código Python que é executado em tempo de execução
- Pode melhorar significativamente o desempenho do seu programa.

```
$ conda install numba
```

Exemplo de migração do código para numba

```
def dot_product(a, b):  
    result = 0  
    for i in range(len(a)):  
        result += a[i] * b[i]  
    return result
```

Exemplo de migração do código para numba

```
def dot_product(a, b):  
    result = 0  
    for i in range(len(a)):  
        result += a[i] * b[i]  
    return result
```

```
import numba  
  
@numba.jit  
def dot_product_numba(a, b):  
    result = 0  
    for i in range(len(a)):  
        result += a[i] * b[i]  
    return result
```



Vamos acelerar essa função com o Numba
Basta adicionar o decorador `@numba.jit` antes da definição da função
Depois de decorar a função chamamos a função normalmente

Funcionamento

- O Numba irá compilar o código durante a primeira execução
- Em iterações subsequentes, executará o código compilado para obter um desempenho otimizado
- Podemos definir
 - Compilação
 - Vetorização
 - Paralelização do código!

Código que funciona

```
from numba import jit
import numpy as np

x = np.arange(100).reshape(10, 10)

@jit(nopython=True) # Set "nopython" mode for best performance, equivalent to
@njit
def go_fast(a): # Function is compiled to machine code when called the first
time
    trace = 0.0
    for i in range(a.shape[0]): # Numba likes loops
        trace += np.tanh(a[i, i]) # Numba likes NumPy functions
    return a + trace # Numba likes NumPy broadcasting

print(go_fast(x))
```


Código que **não** funciona

```
from numba import jit
import pandas as pd

x = {'a': [1, 2, 3], 'b': [20, 30, 40]}

@jit
def use_pandas(a): # Function will not benefit from Numba jit
    df = pd.DataFrame.from_dict(a) # Numba doesn't know about pd.DataFrame
    df += 1                        # Numba doesn't understand what this is
    return df.cov()               # or this!

print(use_pandas(x))
```

Definindo tipos

- O Numba também permite especificar o tipo dos parâmetros e variáveis na função,
- Isso pode melhorar ainda mais o desempenho.
- Aqui estamos mudando a função ``dot_product_numba`` para especificar que os parâmetros ``a`` e ``b`` são matrizes unidimensionais de números de ponto flutuante (``float64``):

-

```
@numba.jit(numba.float64(numba.float64[:],  
numba.float64[:]))  
def dot_product_numba(a, b):  
    result = 0  
    for i in range(len(a)):  
        result += a[i] * b[i]  
    return result
```

Paralelizando

- Ao utilizarmos o decorador @jit do Numba para compilar JIT (Just-in-Time)
- Usamos o argumento parallel=True que indica ao Numba que a função pode ser paralelizada.
- Dentro da função paralelizada, utilizamos o **loop prange** em vez de range para permitir a paralelização.

```
import numba

@numba.jit(parallel=True)
def dot_product_numba(a, b):
    result = 0
    for i in prange(len(a)):
        result += a[i] * b[i]
    return result
```

Funcionamento

- A paralelização é realizada automaticamente pelo Numba, aproveitando os recursos do processador para executar as iterações do loop de forma paralela.
- Lembre-se de que **nem todos os tipos de código são adequados para paralelização**
- **Você deve analisar a consistência, sincronização e condições de corrida, etc.**

Exemplo comparando os desempenhos

```
import numpy as np
import numba as nb
import timeit

# Função em Python puro
def sum_elements_python(arr):
    result = 0.0
    for element in arr:
        result += element
    return result

# Função em Numba com JIT paralelo
@nb.jit
def sum_elements_numba(arr):
    result = 0.0
    for element in nb.prange(len(arr)):
        result += arr[element]
    return result
```

```
@nb.jit(nb.float64(nb.float64[:]))
def sum_elements_numba_type(arr):
    result = 0.0
    for element in nb.prange(len(arr)):
        result += arr[element]
    return result

@nb.jit(fastmath=True)
def sum_elements_numba_fastmath(arr):
    result = 0.0
    for element in nb.prange(len(arr)):
        result += arr[element]
    return result

@nb.jit(parallel=True)
def sum_elements_numba_parallel(arr):
    result = 0.0
    for element in nb.prange(len(arr)):
        result += arr[element]
    return result
```

Exemplo comparando os desempenhos

```
# Dados de entrada
arr = np.random.rand(10000000)
```

```
# Execução em Python puro
python_time = timeit.timeit(lambda: sum_elements_python(arr), number=10)
```

```
# Execução em Numba com JIT
numba_time = timeit.timeit(lambda: sum_elements_numba(arr), number=10)
numba_type_time = timeit.timeit(lambda: sum_elements_numba_type(arr), number=10)
numba_math_time = timeit.timeit(lambda: sum_elements_numba_fastmath(arr), number=10)
numba_par_time = timeit.timeit(lambda: sum_elements_numba_parallel(arr), number=10)
```

```
# Exibe os tempos de execução
print("Python time:", python_time)
print("Numba time:", numba_time)
print("Numba Type time:", numba_type_time)
print("Numba FastMath time:", numba_math_time)
print("Numba Parallel time:", numba_par_time)
```

```
$ python3 ../numba_test.py
Python time: 6.599767065956257
Numba time: 0.14745443197898567
Numba Type time: 0.10892512602731586
Numba FastMath time: 0.07908795500406995
Numba Parallel time: 0.2876081120339222
```

Python com Cython

O que é Cython

- Cython é uma linguagem que é um superconjunto de Python.
- Ela permite que você escreva código Python que é executado como código nativo, o que pode melhorar significativamente a velocidade do seu programa.
- Podemos instalar o suporte
 - `sudo apt install cython3`

Exemplo de migração de código para cython

```
def soma(n):  
    resultado = 0  
    for i in range(1, n+1):  
        resultado += i  
    return resultado
```

Exemplo de migração de código para cython

```
def soma(n):  
    resultado = 0  
    for i in range(1, n+1):  
        resultado += i  
    return resultado
```

```
# Arquivo <soma.pyx>  
def soma_cython(int n):  
    cdef int i  
    cdef int resultado = 0  
    for i in range(1, n+1):  
        resultado += i  
    return resultado
```



Este código é semelhante ao código Python anterior. Mas...

Primeiro, declaramos o tipo dos parâmetros e variáveis usando `cdef`, o que permite que o código seja compilado em linguagem C.

Em seguida, usamos o mesmo loop `for` que a versão Python anterior.

Compilando código cython

- Para compilar o código Cython, precisamos criar um arquivo `setup.py`

```
# Arquivo <setup.py>
from distutils.core import setup
from Cython.Build import cythonize

setup(ext_modules=cythonize("soma.pyx"))
```

Compilando cython

- Para compilar o código, basta executar o seguinte comando no bash:
 - `python setup.py build_ext --inplace`
- Isso criará um arquivo `soma.c` e um arquivo `soma.so` (ou `soma.pyd` no Windows) no diretório atual. O arquivo `.c` é o código C gerado pelo Cython e o arquivo `.so` é o módulo Python compilado.
- Agora, podemos usar a nossa nova função `soma_cython` no nosso código Python:
 - `from soma import soma_cython`
 - `print(soma_cython(1000000))` # Exibe o resultado da soma dos números de 1 a 1.000.000

Exemplo completo

Criando a biblioteca cython

sum_example.pyx

```
def sum_elements_cython(arr):  
    cdef double result = 0.0  
    for element in arr:  
        result += element **2  
    return result
```

setup.py

```
from distutils.core import setup  
from Cython.Build import cythonize  
  
setup(ext_modules=cythonize("sum_example  
.pyx"))
```

Exemplo completo criando uma função python para comparar

`sum_example.pyx`

```
def sum_elements_python(arr):  
    result = 0.0  
    for element in arr:  
        result += element **2  
    return result
```

Exemplo completo

Testando o desempenho das duas versões

```
import numpy as np
import timeit
import pyximport

# Importa a função Cython compilada
pyximport.install()
from sum_example import sum_elements_cython

# Dados de entrada
arr = np.random.rand(1000000)

# Execução em Python puro
python_time = timeit.timeit(lambda: sum_elements_python(arr), number=100)

# Execução em Cython
cython_time = timeit.timeit(lambda: sum_elements_cython(arr), number=100)

# Exibe os tempos de execução
print("Python time:", python_time)
print("Cython time:", cython_time)
```

```
python3 setup.py build_ext -inplace
python3 test.py
Python time: 14.255701961985324
Cython time: 14.738057464011945
```

Python com Dask

Introdução ao Dask

- O que é Dask
 - O Dask é uma biblioteca em Python
 - Facilita a computação paralela e distribuída
 - Executa operações de maneira eficiente e escalável
 - Fornece estruturas de dados flexíveis e operações de alto desempenho
 - Usado principalmente para processamento de dados em larga escala (quando não cabe na memória).

Introdução ao Dask

- Por que usar Dask
 - O objetivo principal do Dask é permitir que os usuários trabalhem com conjuntos de dados maiores do que a capacidade de memória disponível em uma única máquina.
 - Ele divide o processamento em tarefas menores que podem ser executadas em paralelo
 - Pode ser usado em um único computador com vários núcleos ou em um cluster de computadores interconectados.
- Como instalar o Dask?
 - `apt install python3-dask`

Exemplo: Comparação de Desempenho entre NumPy e Dask

```
import numpy as np
import dask.array as da
import time

# Cria um array NumPy de tamanho 100 milhões
arr_np = np.random.random(100_000_000)

# Cria um array Dask a partir do array NumPy
arr_dask = da.from_array(arr_np, chunks=10_000_000)

# Mede o tempo de execução da soma usando NumPy
start_time_np = time.time()
result_np = np.sum(arr_np)
end_time_np = time.time()

# Mede o tempo de execução da soma usando Dask
start_time_dask = time.time()
result_dask = da.sum(arr_dask).compute()
end_time_dask = time.time()
```

```
# Calcula o tempo de execução
time_np = end_time_np - start_time_np
time_dask = end_time_dask - start_time_dask

# Imprime os resultados
print("Resultado NumPy:", result_np)
print("Tempo de execução NumPy:", time_np)
print("Resultado Dask:", result_dask)
print("Tempo de execução Dask:", time_dask)
```

Principais estruturas de dados

- Dask Arrays:
 - São arrays multidimensionais semelhantes aos arrays do NumPy
 - Possui a capacidade de dividir os dados em chunks menores.
 - Isso permite que os cálculos sejam executados de forma paralela em diferentes partes dos dados, aproveitando a capacidade de processamento distribuído.
- Dask DataFrames:
 - São estruturas de dados semelhantes aos DataFrames do Pandas
 - Também com suporte a particionamento e execução paralela.
 - Os Dask DataFrames permitem realizar operações comuns em DataFrames, como filtragem, seleção, agregação e join, de forma escalável e eficiente.

Dask arrays

O que são Dask Arrays

- **São arrays multidimensionais** semelhantes aos arrays do NumPy
- Possui a capacidade de dividir os dados em chunks menores.
- Os chunks são armazenados em diferentes locais da memória ou em dispositivos de armazenamento em disco
- Esses chunks são tratados como blocos independentes de dados, permitindo o processamento paralelo e distribuído.
- Isso permite que os cálculos sejam executados de forma paralela em diferentes partes dos dados, aproveitando a capacidade de processamento distribuído.

Processamento preguiçoso

- Dask Arrays mantêm uma representação simbólica do array completo
- Chamada de grafo computacional, descreve as operações necessárias para realizar cálculos sobre os dados.
- O grafo computacional é construído de forma preguiçosa, ou seja, as operações não são executadas imediatamente, mas sim quando é necessário obter um resultado final.
- Isso permite que o Dask otimize o processamento e minimize a movimentação de dados, evitando a necessidade de carregar todos os dados na memória de uma só vez.

Como criar Dask Arrays

- Existem várias maneiras de criar Dask Arrays.
- 1. Convertendo de um NumPy Array:
 - `numpy_array = np.array([1, 2, 3, 4, 5])`
 - `dask_array = da.from_array(numpy_array, chunks=(2,))`
 - # Chunks, define o tamanho dos blocos de dados.

Como criar Dask Arrays

- 2. Usando a função `da.arange()`:
 - `dask_array = da.arange(1, 100, chunks=(10,))`
 - `# Cria uma sequência de valores dentro do intervalo especificado.`
- 3. Gerando dados aleatórios com a função `da.random.random()`:
 - `dask_array = da.random.random((1000, 1000), chunks=(100, 100))`
 - `# Criando um Dask Array de forma aleatória com dimensões 1000x1000`
- 4. Lendo dados de arquivos com a função `da.from_array()`:
 - `dask_array = da.from_array("dados.csv", chunks=(1000,))`
 - `# Lê dados de arquivos em formatos suportados (como CSV, HDF5, etc.)`

Exemplo: Criação e Manipulação de Dask Arrays

```
•import numpy as np
•import dask.array as da

•# Criação de um Dask Array a partir de uma matriz NumPy
•arr_np = np.arange(1, 101)
•# Cria um Dask Array com chunks de tamanho 10
•arr_dask = da.from_array(arr_np, chunks=(10,))
•Com o Dask Array quebrado em chunks, podemos acessar os elementos individualmente.
•# Acesso aos elementos do Dask Array
•print(arr_dask[0]) # Acessa o primeiro elemento do Dask Array

•# Operações matemáticas com o Dask Array
•arr_doubled = arr_dask * 2 # Multiplica cada elemento por 2
•arr_sum = arr_doubled.sum() # Calcula a soma de todos os elementos

•# Execução dos cálculos com o Dask Array
•result = arr_sum.compute() # Executa os cálculos e retorna o resultado

•print(result) # Imprime o resultado
```

Chunking

- O que é Chunking
- Como Chunking afeta o desempenho
- Como escolher o tamanho dos Chunks
-

Exemplo: Dask array chunks

```
•import dask.array as da

•# Criação de um Dask Array com diferentes tamanhos de chunk
•arr_dask = da.arange(100, chunks=[25, 25, 50])

•# Imprime informações sobre os chunks
•print(arr_dask.chunks)
```

Manipulação de Dask Arrays

- Operações com Dask arrays
 - As operações com o Dask Array são definidas de forma preguiçosa
 - Os cálculos não são executados imediatamente.
 - As operações são adicionadas a um grafo computacional
 - São executadas apenas quando chamamos o método `compute()` para obter os resultados.
- Indexação e Slicing
 - Podemos trabalhar com indexação e slicing de maneira semelhante ao NumPy
 - Acessando e manipulando elementos individuais ou porções específicas dos dados.
 - Essas operações também são preguiçosas

Exemppllo: Indexação e Slicing em dask arrays

```
•import dask.array as da

•# Criação de um Dask Array
•arr = da.arange(1, 101, chunks=(10,))

•# Indexação em um Dask Array
•print(arr[0].compute()) # Acessa o primeiro elemento
•print(arr[5:15].compute()) # Acessa um intervalo de elementos
•print(arr[[1, 3, 8]].compute()) # Acessa elementos específicos

•# Slicing em um Dask Array
•print(arr[:10].compute()) # Acessa os primeiros 10 elementos
•print(arr[20:].compute()) # Acessa todos os elementos a partir do índice 20
•print(arr[:,2].compute()) # Acessa todos os elementos com um passo de 2
```

Manipulação de Dask Arrays

- Operações Matemáticas (preguiçosas)

- `result = arr1 + arr2`
- `result = arr2 - arr1`
- `result = arr2 * arr1`
- `result = arr2 / arr1`
- `result = arr2 ** 2`

- Redução de Dados (preguiçosas)

- `result = arr.sum()`
- `result = arr.min()`
- `result = arr.max()`
- `result = arr.mean()`
- `result = arr.std()`

Exemplo: Manipulação de um Dask Array com Operações Matemáticas

```
•import dask.array as da

•# Criação de um Dask Array
•arr_dask = da.arange(1, 11, chunks=(5,))

•# Operações matemáticas com o Dask Array
•arr_squared = arr_dask ** 2 # Eleva cada elemento ao quadrado
•arr_sum = arr_squared.sum() # Calcula a soma de todos os elementos

•# Execução dos cálculos com o Dask Array
•result = arr_sum.compute() # Executa os cálculos e retorna o resultado

•print(result) # Imprime o resultado
```


exemplo: manipulação de um Dask Array com redução de dados

```
•import dask.array as da

•# Criação de um Dask Array
•arr_dask = da.arange(1, 101, chunks=(10,))

•# Redução de dados com o Dask Array
•arr_sum = arr_dask.sum() # Calcula a soma de todos os elementos
•arr_min = arr_dask.min() # Encontra o valor mínimo
•arr_max = arr_dask.max() # Encontra o valor máximo

•# Execução dos cálculos com o Dask Array
•result_sum = arr_sum.compute() # Executa a soma e retorna o resultado
•result_min = arr_min.compute() # Executa a busca pelo valor mínimo e retorna o resultado
•result_max = arr_max.compute() # Executa a busca pelo valor máximo e retorna o resultado

•print("Soma:", result_sum) # Imprime a soma
•print("Mínimo:", result_min) # Imprime o valor mínimo
•print("Máximo:", result_max) # Imprime o valor máximo
```

Data frames

Dask DataFrames

- O que são DataFrames?
 - São estruturas de dados tabulares em semelhantes aos DataFrames do Pandas
 - DataFrames organizam os dados em colunas nomeadas e fornecem uma maneira conveniente de realizar operações de manipulação, agregação e análise de dados tabulares.
 - DataFrames dividem os dados em chunks menores
 - Também com suporte a particionamento e execução paralela.
 - Os Dask DataFrames permitem realizar operações comuns em DataFrames, como filtragem, seleção, agregação e join, de forma escalável e eficiente.
 - O processamento de DataFrames em Dask também acontece de forma preguiçosa

Como criar Dask DataFrames

Existem várias maneiras de criar Dask DataFrames.

1. Convertendo de um Pandas DataFrame:

```
import dask.dataframe as dd
import pandas as pd
pandas_df = pd.DataFrame({'A': [1, 2, 3, 4, 5],
                           'B': ['a', 'b', 'c', 'd', 'e']})
dask_df = dd.from_pandas(pandas_df, npartitions=2)
# npartitions é o número de chunks em que os dados serão divididos.
```

Como criar Dask DataFrames

2. Lendo dados de arquivos com a função `dd.read_csv()`:

```
import dask.dataframe as dd
dask_df = dd.read_csv('dados.csv')
```

3. Criando um Dask DataFrame com dados gerados:

```
import dask.dataframe as dd
dask_df = dd.from_array([(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e')], columns=['A', 'B'])
#dd.from_array() cria um Dask DataFrame a partir de uma lista de tuplas ou arrays
```

Exemplo: Criação e Manipulação de um Dask DataFrame

```
•import dask.dataframe as dd
•import pandas as pd

•# Criação de um Dask DataFrame a partir de dados existentes
•data = {'A': [1, 2, 3, 4, 5],
•        'B': ['a', 'b', 'c', 'd', 'e'],
•        'C': [10.5, 20.2, 30.7, 40.1, 50.9]}
•df = dd.from_pandas(pd.DataFrame(data), npartitions=2)

•# Visualização dos primeiros registros do Dask DataFrame
•print(df.head())

•# Filtragem de dados
•filtered_df = df[df['A'] > 2]
•print(filtered_df.head())

•# Adição de uma nova coluna
•df['D'] = df['A'] * df['C']
•print(df.head())

•# Agregação de dados
•aggregated_df = df.groupby('B').sum()
•print(aggregated_df.head())

•# Computação de resultados
•result = aggregated_df.compute()
•print(result)
```

Manipulação de Dask DataFrames

- Seleção de Dados
- Agrupamento de Dados
- Junção de Dados
-

exemplo: Manipulação de um Dask DataFrame com Agrupamento e Junção de Dados

```
•import dask.dataframe as dd
•import pandas as pd

•# Criação dos Dask DataFrames
•df1 = dd.from_pandas(pd.DataFrame({'A': [1, 2, 3, 4, 5],
•                                     'B': ['a', 'b', 'c', 'd', 'e']}), npartitions=2)
•df2 = dd.from_pandas(pd.DataFrame({'B': ['a', 'b', 'c', 'd', 'e'],
•                                     'C': [10, 20, 30, 40, 50]}), npartitions=2)

•# Operação de junção (join) dos DataFrames
•joined_df = df1.merge(df2, on='B')

•# Agrupamento de dados
•grouped_df = joined_df.groupby('B').sum()

•# Visualização dos resultados
•print(grouped_df.compute())
```


Computação distribuída com dask

O que é Computação Distribuída

- Uma abordagem para processar grandes volumes de dados
- Executar tarefas complexas de forma eficiente
- Dividindo o processamento entre vários recursos computacionais, como computadores, servidores ou clusters de máquinas.

Como Dask faz Computação Distribuída

1. Divisão de dados em partições: Divide os dados em pequenos pedaços
2. Grafo computacional: Usado para rastrear as dependências entre as tarefas e as partições de dados
3. Agendamento de tarefas: Analisa o grafo computacional e decide quais tarefas podem ser executadas em paralelo e quais dependem de outras tarefas.
4. Integração com outros ecossistemas: como o Apache Hadoop e o Apache Spark
5. Adaptação ao tamanho do cluster: flexível e adaptável ao tamanho do cluster disponível

•

computação distribuída com o Dask utilizando múltiplas máquinas

- Configuração do Cluster:
 - Certifique-se de ter o Dask instalado em todas as máquinas que deseja usar.
 - Em uma máquina principal (chamada de "nó mestre"), inicie o cluster Dask utilizando o comando `dask-scheduler`. Isso irá iniciar o agendador do Dask e fornecerá um endereço IP e uma porta para o cluster.
 - Em cada máquina de trabalho (chamada de "nó trabalhador"), inicie um nó trabalhador Dask utilizando o comando `dask-worker` `<endereço_IP_do_nó_mestre>:<porta_do_agendador>`. Isso irá conectar o nó trabalhador ao nó mestre e permitir que ele participe da computação distribuída.

Certifique-se de substituir `<endereço_IP_do_nó_mestre>` pela endereço IP do nó mestre e `<porta_do_agendador>` pela porta na qual o agendador do Dask está rodando.

computação distribuída com o Dask utilizando múltiplas máquinas

- Código de Computação Distribuída:
 - Utilize as estruturas de dados distribuídas do Dask, como Dask Arrays ou Dask DataFrames, para realizar operações paralelas em seus dados.
 - Ao criar suas estruturas de dados distribuídas, especifique o tamanho das partições (chunks) que serão distribuídas entre os nós do cluster.
 - Execute as operações em suas estruturas de dados distribuídas como faria normalmente no Dask.
 - Use a função `compute()` para acionar a computação distribuída e obter os resultados.

Escalabilidade com dask

- Algumas estratégias para escalar com o Dask:
- **Aumentar o tamanho dos chunks:** Aumentar o tamanho dos chunks pode melhorar o desempenho ao reduzir a sobrecarga de comunicação entre os nós do cluster. No entanto, é importante encontrar um equilíbrio, pois chunks muito grandes podem levar a problemas de memória.
- **Adicionar mais nós trabalhadores:** Quanto mais nós trabalhadores você tiver, mais operações poderão ser executadas simultaneamente, resultando em um processamento mais rápido.
- **Usar um cluster de computação em nuvem:** Se você não possui um cluster próprio, pode aproveitar serviços de computação em nuvem que oferecem recursos escaláveis.
- **Aproveitar a paralelização automática:** O Dask tem a capacidade de paralelizar automaticamente as operações sempre que possível. Ele identifica as dependências entre as tarefas e executa aquelas que não têm dependências simultaneamente. Ao aproveitar essa paralelização automática, você pode obter melhorias de desempenho sem precisar ajustar manualmente o código ou a configuração.
- **Otimizar as operações:** O Dask oferece várias otimizações e estratégias para melhorar o desempenho das operações. Isso inclui técnicas como filtragem antecipada (early filtering), fusão de operações (operation fusion) e redução de comunicação. Ao utilizar essas otimizações, você pode obter um desempenho mais eficiente e escalável.

Exemplo: Escalabilidade com Dask e Kubernetes

- Configuração do cluster Kubernetes:
 - Configure um cluster Kubernetes com os nós desejados, seja localmente ou em um provedor de nuvem.
 - Certifique-se de que o ambiente do Kubernetes esteja configurado corretamente e você tenha acesso ao cluster.
- Configuração do ambiente Dask:
 - Instale o pacote `dask-kubernetes` para integrar o Dask com o Kubernetes.
 - Importe os módulos necessários e configure o cluster Kubernetes no Dask.
- Criação e execução da computação:
 - Crie um objeto Cluster do Dask usando a classe `KubeCluster` do `dask_kubernetes`.
 - Ajuste as configurações do cluster, como o número de nós trabalhadores, tamanho das instâncias e outras opções específicas do Kubernetes.
 - Crie e execute suas tarefas distribuídas usando as estruturas de dados do Dask, como `Dask Arrays` ou `Dask DataFrames`.

Exemplo: Escalabilidade com Dask e Kubernetes

```
•from dask_kubernetes import KubeCluster
•from dask.distributed import Client

•# Configuração do cluster Kubernetes
•cluster = KubeCluster()
•cluster.scale(10) # Definindo o número de nós trabalhadores desejados
•cluster.adapt(minimum=2, maximum=20) # Configurando o escalonamento automático

•# Criação do cliente Dask para interagir com o cluster
•client = Client(cluster)

•# Criação e execução da computação distribuída
•# Exemplo usando um Dask Array
•import dask.array as da
•x = da.random.random((10000, 10000), chunks=(1000, 1000))
•y = x.mean(axis=0)
•result = y.compute()

•print(result)
```